

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности
Высшая школа технологий и искусственного интеллекта
Направление: 02.03.01 Математика и компьютерные науки

Теоретические основы баз данных
Отчет по курсовой работе

Реализация базы данных для предметной области
«Учёт движения городского транспорта»

Выполнил:

Студент, группы
5130201/40001

_____ Ови Мд Шамин Ясир

Проверил:

Преподаватель,

_____ Попов Сергей Геннадьевич

«_____» _____ 20__ г.

Санкт-Петербург, 2026

Реферат

Предметной областью данной курсовой работы является учет движения городского общественного транспорта. Основной проблемой является необходимость структурированного хранения информации о транспортных средствах, маршрутах, остановках, водителях и выполненных рейсах. В ходе выполнения работы была проведена анализ предметной области, определены основные сущности и их взаимосвязи, разработана ER-модель и реализована схема реляционной базы данных.

Для реализации базы данных использовалась система управления базами данных PostgreSQL. Были разработаны SQL-скрипты для создания структуры базы данных, генерации тестовых данных и выполнения аналитических запросов.

Разработанная база данных может использоваться для хранения информации о рейсах, мониторинга активности маршрутов и формирования аналитических отчетов о работе городского транспорта.

Содержание

Реферат	2
1 Введение	4
2 Постановка задачи	5
2.1 Аналитика работы Учёт движения городского транспорта	6
2.2 Цели создания базы данных	8
2.3 Описание сущностей	8
2.4 Описание ролей	9
2.5 ER-диаграмма	10
2.5.1 Чтение ER-диаграммы	12
2.6 Схема связи объектов	13
3 Проектирование	15
3.1 Лингвистическое описание	15
3.2 Таблицы метаданных	16
3.3 Схема базы данных на русском языке	23
3.4 Генерация данных	26
4 Программирование	27
4.1 Запрос 1	27
4.2 Запрос 2	30
4.3 Запрос 3	33
4.4 Запрос 4	36
4.5 Запрос 5	39
4.6 Запрос 6	43
4.7 Запрос 7	47
4.8 Запрос 8	51
4.9 Запрос 9	55
Заключение	58
Список использованной литературы	59
Приложение А. Исходный код программы генерации схемы базы данных	60
Приложение Б. Исходный код программы заполнения базы данных текстовыми данными	63

1 Введение

Городской общественный транспорт представляет собой сложную систему, включающую различные виды транспортных средств, фиксированные маршруты, остановки, водителей и ежедневные рейсы. Для такой системы важно обеспечить надежное хранение структурированной информации, поскольку транспортная деятельность сопровождается большим количеством взаимосвязанных данных: транспортные средства закрепляются за маршрутами, водители выполняют рейсы, а остановки связаны с маршрутами в определенной последовательности.

База данных представляет собой организованную совокупность данных, предназначенную для хранения, обработки и извлечения информации в рамках определенной предметной области. Системы управления базами данных позволяют обеспечивать целостность данных, уменьшать их дублирование, контролировать взаимосвязи между записями и оперативно получать необходимую информацию для решения практических задач.

В данной курсовой работе рассматривается предметная область учета движения городского общественного транспорта. Актуальность темы обусловлена необходимостью систематизации информации о транспортных средствах, маршрутах, остановках и выполненных рейсах. Без структурированной базы данных поддержание согласованности такой информации становится затруднительным, особенно при большом объеме данных.

Целью работы является разработка и реализация реляционной базы данных для хранения информации о движении городского транспорта. База данных должна описывать основные сущности предметной области, определять связи между ними, обеспечивать генерацию тестовых данных и поддерживать выполнение SQL-запросов для анализа.

В качестве средств реализации использовалась система управления базами данных PostgreSQL и SQL-скрипты. Проект включает схему базы данных, сгенерированные тестовые данные, а также набор запросов, демонстрирующих выборку данных, соединения таблиц, группировку, агрегирование, операции над множествами и аналитическую обработку данных.

2 Постановка задачи

Целью данной курсовой работы является разработка реляционной базы данных для учёта движения городского общественного транспорта, а также реализация заданных запросов.

Для достижения поставленной цели необходимо выполнить следующие задачи:

1. Проанализировать предметную область учёта движения городского транспорта.
2. Составить перечень ключевых сущностей, их атрибутов и ролей на основе предметной области.
3. Составить ER-диаграмму в нотации Чена.
4. Составить схему связей сущностей.
5. Спроектировать схему базы данных.
6. Составить таблицы метаданных.
7. Сделать лингвистическое описание базы данных.
8. Составить диаграмму порядка создания таблиц базы данных.
9. Воссоздать базу данных в системе управления «PostgreSQL».
10. Разработать программу заполнения для базы данных.
11. Реализовать оптимальные запросы по заданному описанию.

2.1 Аналитика работы Учёт движения городского транспорта

Описание предметной области:

Предметная область данной работы связана с функционированием системы городского общественного транспорта и учётом движения транспортных средств внутри города. Городской общественный транспорт является важной частью городской инфраструктуры и обеспечивает регулярное транспортное сообщение между различными районами города за счёт организованного движения транспортных средств по установленным маршрутам.

С организационной точки зрения городской транспорт можно рассматривать как систему, включающую транспортные средства, маршруты, водителей и элементы инфраструктуры, обеспечивающие их работу. Водитель является участником процесса выполнения рейсов и осуществляет управление транспортным средством на маршруте. Правильная организация и учёт движения транспорта позволяют определить степень загрузки маршрутов и распределение транспортных средств.

В рамках данной работы городской транспорт рассматривается как операционная система, основной функцией которой является выполнение маршрутов транспортными средствами под управлением водителей и регистрация этих выполнений в виде записей о движении.

В предметной области можно выделить процесс выполнения рейса. Данный процесс включает назначение транспортного средства и водителя на маршрут, выполнение движения по маршруту с остановками и фиксацию результата в виде записи о рейсе. Участниками процесса являются водитель и транспортное средство.

Вид транспорта представляет собой классификацию транспортных средств по их техническим и эксплуатационным характеристикам. В системе городского общественного транспорта используются различные виды транспорта в зависимости от инфраструктуры, вместимости и условий эксплуатации.

В рассматриваемой предметной области выделяются следующие виды транспорта:

- автобус;
- трамвай;
- троллейбус.

Каждый вид транспорта определяет характеристики транспортных средств, которые к нему относятся. Например, автобусы передвигаются по городским дорогам, трамваи движутся по рельсам, а троллейбусы используют контактную электрическую сеть. Эти особенности влияют на назначение транспортных средств на маршруты.

Транспортные средства представляют собой физические единицы транспорта, выполняющие движение по маршрутам. Каждое транспортное средство относится к определённому виду транспорта и имеет уникальный идентификатор. Транспортные средства используются многократно в течение рабочего дня и могут выполнять несколько рейсов. Одно и то же транспортное средство может выполнять один и тот же маршрут несколько раз или использоваться на разных маршрутах.

Водители являются участниками процесса движения транспорта. Каждый водитель управляет транспортным средством и выполняет рейсы по заданному маршруту. Один водитель может выполнять несколько рейсов в течение дня и может быть закреплён за различными транспортными средствами.

Транспортные маршруты являются одним из основных элементов организации движения транспорта. Маршрут представляет собой фиксированный путь движения, определяющий последовательность перемещения транспортного средства. Каждый маршрут имеет уникальный идентификатор и устойчивую структуру. В течение дня один маршрут может выполняться многократно различными транспортными средствами.

Остановки представляют собой точки на маршруте, в которых транспортное средство выполняет остановку. Остановки формируют структуру маршрута и определяют последовательность движения. Одна остановка может входить в состав нескольких маршрутов, а один маршрут включает несколько остановок.

Связь между маршрутами и остановками имеет тип «многие ко многим». Для её реализации используется промежуточная сущность ROUTE_STOP, которая определяет состав маршрута и порядок следования остановок.

Рейсы являются основной операционной сущностью предметной области. Рейс представляет собой выполнение маршрута транспортным средством под управлением водителя в определённый момент времени. Каждый рейс характеризуется транспортным средством, маршрутом, водителем и временем выполнения. В течение дня транспортное средство может выполнять несколько рейсов, а маршруты могут выполняться многократно.

Временной аспект движения транспорта отражает распределение рейсов во времени. Учёт времени позволяет определить интенсивность движения транспорта в различные периоды.

Результат анализа предметной области заключается в формировании структурированной модели движения городского транспорта, включающей транспортные средства, водителей, маршруты, остановки и рейсы. Данная модель соответствует требованиям курсовой работы и служит основой для проектирования базы данных.

2.2 Цели создания базы данных

1. Формирование базы данных транспортных средств, их видов и характеристик.
2. Фиксация информации о маршрутах, остановках и порядке их следования.
3. Регистрация данных о выполненных рейсах, водителях и маршрутно-остановочных связях.

2.3 Описание сущностей

- **Вид транспорта** — это классификация транспортных средств, определяющая их технические характеристики, условия эксплуатации и особенности использования в системе городского транспорта. Атрибуты: наименование вида транспорта, описание, тип инфраструктуры, вместимость, источник энергии.
- **Источник энергии** — это классификационная сущность, определяющая тип используемой энергии для функционирования транспортных средств, что влияет на их технические характеристики и условия эксплуатации. Атрибуты: наименование источника энергии.
- **Транспортное средство** — это физическая единица транспорта, предназначенная для выполнения перевозок по маршрутам и относящаяся к определённому виду транспорта. Атрибуты: государственный номер, модель транспортного средства, вместимость, инвентарный номер, год выпуска.
- **Водитель** — это участник системы городского транспорта, который управляет транспортным средством и выполняет рейсы по установ-

ленным маршрутам. Атрибуты: фамилия, имя, отчество, категория удостоверения, табельный номер.

- **Маршрут** — это установленный путь движения транспортного средства, определяющий последовательность перемещения и соединяющий остановки в пределах городской инфраструктуры. Атрибуты: номер, длина, название, тип маршрута, статус.
- **Рейс** — это выполнение маршрута транспортным средством под управлением водителя в определённую дату с фиксированным временем отправления и прибытия. Атрибуты: номер рейса, дата, время отправления, время прибытия, статус.
- **Остановка** — это определённое место на маршруте, где транспортное средство выполняет остановку для посадки и высадки пассажиров. Атрибуты: наименование остановки, координаты остановки, тип остановки, адрес остановки, район расположения.
- **Маршрут-остановка** — это промежуточная сущность, отражающая связь между маршрутом и остановкой, а также порядок следования остановок на маршруте. Атрибуты: номер записи, номер маршрута, наименование остановки, порядковый номер остановки, время прибытия, расстояние от начала маршрута.

2.4 Описание ролей

Водитель является участником процесса выполнения рейса и отвечает за управление транспортным средством на маршруте. Он соблюдает установленный график движения и обеспечивает безопасное выполнение перевозки пассажиров.

Транспортное средство используется для выполнения перевозок и участвует в процессе выполнения рейсов. Оно обеспечивает движение по маршруту и является основным средством транспортировки в системе городского транспорта.

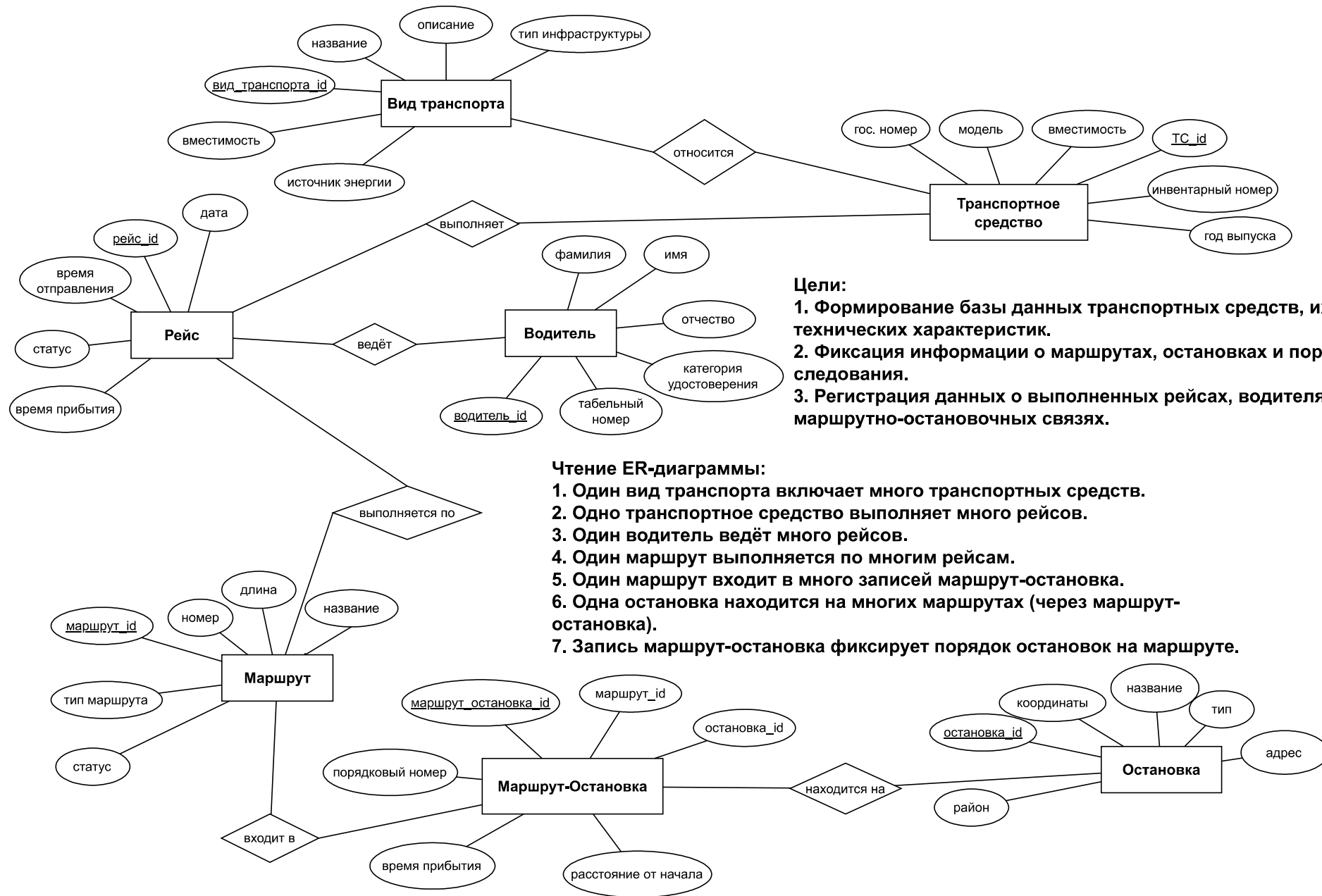
Маршрут определяет направление движения транспортного средства и последовательность прохождения остановок. Он используется при организации движения и задаёт структуру выполнения рейсов в транспортной системе.

Рейс представляет собой процесс выполнения маршрута транспортным средством под управлением водителя. Он фиксируется по времени выполнения и отражает факт выполнения перевозки в системе.

2.5 ER-диаграмма

На рисунке 1 представлена ER-диаграмма предметной области «Учёт движения городского транспорта».

Рисунок 1 — ER-диаграмма предметной области «Учёт движения городского транспорта»



2.5.1 Чтение ER-диаграммы

1. Один вид транспорта включает много транспортных средств.
2. Одно транспортное средство выполняет много рейсов.
3. Один водитель ведёт много рейсов.
4. Один рейс выполняется по одному маршруту.
5. Один маршрут включает много рейсов.
6. Один маршрут входит в много записей маршрут-остановка.
7. Одна остановка находится на многих маршрутах (через маршрут-остановка).
8. Запись маршрут-остановка фиксирует порядок остановок на маршруте.

2.6 Схема связи объектов

На рисунке 2 представлена схема связи объектов предметной области «Учёт движения городского транспорта».

Рисунок 2

Схема связи объектов

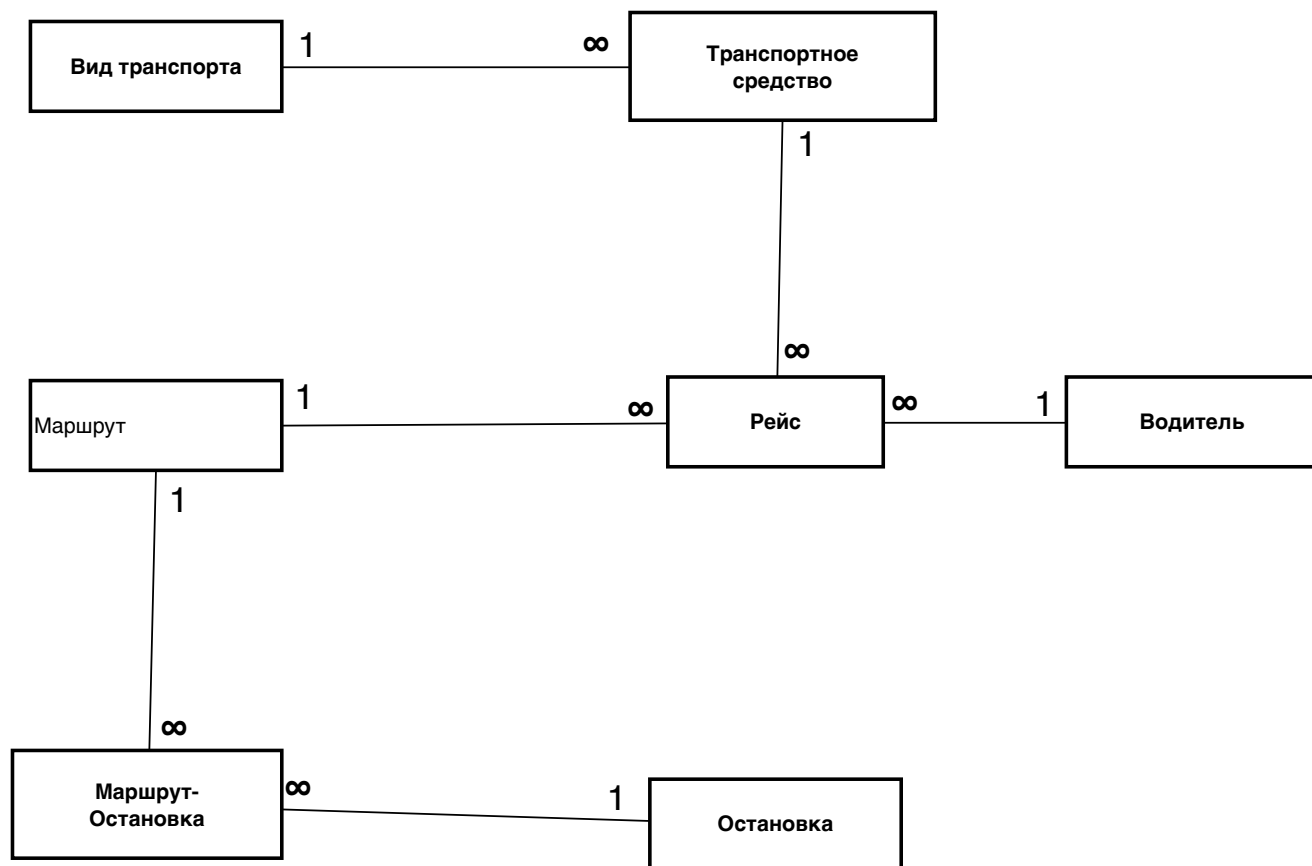


Рис. 2. Схема связи объектов
Связи «один ко многим» изображены символами «1» и «∞».

3 Проектирование

3.1 Лингвистическое описание

- **Вид_транспорта**
(<id_Вид_транспорта, VARCHAR(50)>, <Описание, VARCHAR(200)>, <Тип_инфраструктуры, VARCHAR(50)>, <Источник_энергии, VARCHAR(50)>, <Вместимость, INT>)
- **Источник_энергии**
(<id_Источник_энергии, VARCHAR(50)>, <Наименование, VARCHAR(100)>)
- **Транспортное_средство**
(<id_Транспортное_средство, VARCHAR(20)>, <Модель_транспортного_средства, VARCHAR(100)>, <Вместимость, INT>, <Инвентарный_номер, VARCHAR(20)>, <Год_выпуска, INT>, <id_Вид_транспорта, VARCHAR(50)>)
- **Водитель**
(<id_Водитель, INT>, <Фамилия, VARCHAR(100)>, <Имя, VARCHAR(100)>, <Отчество, VARCHAR(100)>, <Категория_удостоверения, VARCHAR(10)>)
- **Модель_транспортного_средства**
(<id_Модель_транспортного_средства, VARCHAR(50)>, <Наименование, VARCHAR(100)>)
- **Маршрут**
(<id_Маршрут, INT>, <Название, VARCHAR(100)>, <Длина, DECIMAL(6,2)>, <Тип_маршрута, VARCHAR(50)>, <Статус, VARCHAR(20)>)
- **Рейс**
(<id_Рейс, INT>, <Дата, DATE>, <Время_отправления, TIME>, <Время_прибытия, TIME>, <Статус, VARCHAR(20)>, <id_Транспортное_средство, VARCHAR(20)>, <id_Водитель, INT>, <id_Маршрут, INT>)
- **Остановка**
(<id_Остановка, VARCHAR(100)>, <Адрес_остановки, VARCHAR(200)>, <Тип_остановки, VARCHAR(50)>, <Район_расположения, VARCHAR(100)>, <Координаты_остановки, VARCHAR(50)>)
- **Маршрут_Остановка**
(<id_Маршрут_Остановка, INT>, <id_Маршрут, INT>, <id_Остановка, VARCHAR(100)>, <Порядковый_номер_остановки, INT>, <Расстояние_от_начала_маршрута, DECIMAL(6,2)>, <Время_прибытия, TIME>)

3.2 Таблицы метаданных

	TRANSPORT_TYPE	ВИД ТРАНСПОРТА		
	Название _ENG	Название _RUS	Тип атрибута	NULL/ NOT NULL
PK	id_Transport_Type	id_Вид_транспорта	VARCHAR(50)	NOT NULL
	Description	Описание	VARCHAR(200)	NULL
	Infrastructure_Type	Тип_инфраструктуры	VARCHAR(50)	NULL
	Energy_Source	Источник_энергии	VARCHAR(50)	NULL
	Capacity	Вместимость	INT	NULL

Рис. 3. Таблица «Вид транспорта»

Первичный ключ **id_Transport_Type** имеет тип **VARCHAR(50)** и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и не может содержать пустые значения. Атрибут **Description** имеет тип **VARCHAR(200)**, поскольку предназначен для хранения текстовой информации переменной длины и может быть необязательным. Атрибуты **Infrastructure_Type** и **Energy_Source** имеют тип **VARCHAR(50)**, так как содержат короткие текстовые характеристики, для которых данного размера достаточно. Атрибут **Capacity** имеет тип **INT**, поскольку хранит целочисленное значение, что позволяет выполнять операции сравнения и обработки данных. Использование данных типов соответствует стандартным рекомендациям по выбору типов данных в СУБД.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

	ENERGY_SOURCE	ИСТОЧНИК_ЭНЕ			
	Название _ENG	Название _RUS	Тип атрибута	NULL/ NOT NULL	Примечание
PK	id_Energy_Source	id_Источник_энергии	VARCHAR(50)	NOT NULL	
	Name	Наименование	VARCHAR(100)	NOT NULL	

Рис. 10. Таблица «Источник энергии»

Первичный ключ **id_Energy_Source** имеет тип **VARCHAR(50)** и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и может хранить содержательные текстовые значения. Атрибут **Name** имеет тип **VARCHAR(100)** и ограничение **NOT NULL**, поскольку представляет обязательное текстовое наименование источника энергии. Использование символьных типов данных соответствует их классификации в СУБД :contentReference[oaicite:0]index=0.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

	VEHICLE	ТРАНСПОРТНОЕ СРЕДСТВО			
	Название ENG	Название RUS	Тип атрибута	NULL/ NOT NULL	Примечание
PK	id_Vehicle	id_Транспортное_средство	VARCHAR(20)	NOT NULL	
	Vehicle_Model	Модель транспортного средства	VARCHAR(100)	NULL	
	Capacity	Вместимость	INT	NULL	
	Inventory_Number	Инвентарный_номер	VARCHAR(20)	NULL	
	Year_of_Manufacture	Год_выпуска	INT	NULL	
FK	id_Transport_Type	id_Вид транспорта	VARCHAR(50)	NOT NULL	Внешний ключ → id_Transport_Type

Рис. 4. Таблица «Транспортное средство»

Первичный ключ **id_Vehicle** имеет тип **VARCHAR(20)** и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и не может содержать пустые значения. Атрибут **Vehicle_Model** имеет тип **VARCHAR(100)**, поскольку предназначен для хранения текстового наименования транспортного средства переменной длины. Атрибут **Capacity** имеет тип **INT**, так как отражает количественную характеристику и позволяет выполнять операции сравнения и обработки данных. Атрибут **Inventory_Number** имеет тип **VARCHAR(20)**, поскольку представляет собой текстовый идентификатор, используемый внутри организации. Атрибут **Year_of_Manufacture** имеет тип **INT**, так как хранит год выпуска транспортного средства. Внешний ключ **id_Transport_Type** имеет тип **VARCHAR(50)** и ограничение **NOT NULL**, поскольку устанавливает обязательную связь с таблицей видов транспорта и обеспечивает ссылочную целостность данных. Использование данных типов соответствует классификации типов данных (числовые и символьные) в СУБД :contentReference[oaicite:0]index=0.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

	VEHICLE_MODEL	МОДЕЛЬ ТРАНСПОРТА			
	Название ENG	Название RUS	Тип атрибута	NULL/ NOT NULL	Примечание
PK	id_Vehicle_Model	id_Модель_транспорта	VARCHAR(50)	NOT NULL	
	Name	Наименование	VARCHAR(100)	NOT NULL	

Рис. 11. Таблица «Модель транспорта»

Первичный ключ **id_Vehicle_Model** имеет тип **VARCHAR(50)**

и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и может хранить содержательные текстовые значения. Атрибут **Name** имеет тип **VARCHAR(100)** и ограничение **NOT NULL**, поскольку представляет обязательное текстовое наименование модели транспортного средства. Использование символьных типов данных соответствует их классификации в СУБД :contentReference[oaicite:0]index=0.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

	DRIVER	ВОДИТЕЛЬ			
	Название _ENG	Название _RUS	Тип атрибута	NULL/ NOT NULL	Примечание
PK	id_Driver	id_Водитель	INT	NOT NULL	
	Last_Name	Фамилия	VARCHAR(100)	NOT NULL	
	First_Name	Имя	VARCHAR(100)	NOT NULL	
	Middle_Name	Отчество	VARCHAR(100)	NULL	
	License_Category	Категория удостоверения	VARCHAR(10)	NOT NULL	

Рис. 5. Таблица «Водитель»

Первичный ключ **id_Driver** имеет тип **INT** и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и обеспечивает эффективную обработку числовых данных. Атрибуты **Last_Name** и **First_Name** имеют тип **VARCHAR(100)** и ограничение **NOT NULL**, поскольку являются обязательными текстовыми характеристиками водителя. Атрибут **Middle_Name** имеет тип **VARCHAR(100)** и допускает значение **NULL**, так как отчество может отсутствовать. Атрибут **License_Category** имеет тип **VARCHAR(10)** и ограничение **NOT NULL**, поскольку хранит краткое текстовое обозначение категории удостоверения. Использование числовых и символьных типов данных соответствует их классификации в СУБД :contentReference[oaicite:0]index=0.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

	ROUTE	МАРШРУТ			
	Название ENG	Название RUS	Тип атрибута	NULL/ NOT NULL	Примечание
PK	id_Route	id_Маршрут	INT	NOT NULL	
	Route_Name	Название	VARCHAR(100)	NOT NULL	
	Length_km	Длина	DECIMAL(6,2)	NULL	
	Route_Type	Тип_маршрута	VARCHAR(50)	NULL	
	Status	Статус	VARCHAR(20)	NOT NULL	

Рис. 6. Таблица «Маршрут»

Первичный ключ **id_Route** имеет тип **INT** и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и обеспечивает эффективную обработку числовых данных. Атрибут **Route_Name** имеет тип **VARCHAR(100)** и ограничение **NOT NULL**, поскольку представляет обязательное текстовое наименование маршрута. Атрибут **Length_km** имеет тип **DECIMAL(6,2)**, так как хранит числовое значение с дробной частью, что обеспечивает необходимую точность измерений. Атрибут **Route_Type** имеет тип **VARCHAR(50)**, поскольку содержит текстовое классификационное значение и может быть необязательным. Атрибут **Status** имеет тип **VARCHAR(20)** и ограничение **NOT NULL**, так как отражает обязательное состояние маршрута. Использование числовых и символьных типов данных соответствует их классификации в СУБД: contentReference[oaicite:0]index=0.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

	TRIP	РЕЙС			
	Название ENG	Название RUS	Тип атрибута	NULL/ NOT NULL	Примечание
PK	id_Trip	id_Рейс	INT	NOT NULL	
	Trip_Date	Дата	DATE	NOT NULL	
	Departure_Time	Время_отправления	TIME	NOT NULL	
	Arrival_Time	Время_прибытия	TIME	NULL	
	Status	Статус	VARCHAR(20)	NOT NULL	
FK	id_Vehicle	id_Транспортное_средство	VARCHAR(20)	NOT NULL	Внешний ключ → id_Vehicle
FK	id_Driver	id_Водитель	INT	NOT NULL	Внешний ключ → id_Driver
FK	id_Route	id_Маршрут	INT	NOT NULL	Внешний ключ → id_Route

Рис. 7. Таблица «Рейс»

Первичный ключ **id_Trip** имеет тип **INT** и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и обеспечивает эффективную обработку числовых данных. Атрибут

Trip_Date имеет тип **DATE** и ограничение **NOT NULL**, поскольку хранит обязательное значение даты выполнения рейса. Атрибуты **Departure_Time** и **Arrival_Time** имеют тип **TIME**, так как предназначены для хранения временных значений, при этом **Arrival_Time** может быть необязательным. Атрибут **Status** имеет тип **VARCHAR(20)** и ограничение **NOT NULL**, поскольку хранит краткое текстовое описание состояния рейса. Внешние ключи **id_Vehicle**, **id_Driver** и **id_Route** имеют типы, соответствующие первичным ключам связанных таблиц, и обеспечивают ссылочную целостность данных. Использование числовых, символьных и временных типов данных соответствует их классификации в СУБД :contentReference[oaicite:0]index=0.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

	STOP	ОСТАНОВКА			
	Название ENG	Название RUS	Тип атрибута	NULL/ NOT NULL	Примечание
PK	id_Stop	id_Остановка	VARCHAR(100)	NOT NULL	
	Stop_Address	Адрес_остановки	VARCHAR(200)	NULL	
	Stop_Type	Тип_остановки	VARCHAR(50)	NULL	
	District	Район_расположения	VARCHAR(100)	NULL	
	Coordinates	Координаты остановки	VARCHAR(50)	NULL	

Рис. 8. Таблица «Остановка»

Первичный ключ **id_Stop** имеет тип **VARCHAR(100)** и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и может хранить содержательные текстовые значения. Атрибут **Stop_Address** имеет тип **VARCHAR(200)**, поскольку предназначен для хранения текстовой информации переменной длины и может быть необязательным. Атрибут **Stop_Type** имеет тип **VARCHAR(50)**, так как содержит краткое классификационное значение. Атрибут **District** имеет тип **VARCHAR(100)**, поскольку хранит текстовое наименование района. Атрибут **Coordinates** имеет тип **VARCHAR(50)**, так как представляет координаты в текстовом формате. Использование символьных типов данных соответствует их классификации в СУБД :contentReference[oaicite:0]index=0.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

	ROUTE_STOP	МАРШРУТ ОСТАНОВКА			
	Название ENG	Название RUS	Тип атрибута	NULL/ NOT NULL	Примечание
PK	id_Route_Stop	id_Маршрут_Остановка	INT	NOT NULL	
FK	id_Route	id_Маршрут	INT	NOT NULL	Внешний ключ → id_Route
FK	id_Stop	id_Остановка	VARCHAR(100)	NOT NULL	Внешний ключ → id_Stop
	Stop_Order	Порядковый_номер остановки	INT	NOT NULL	
	Distance_From_Start	Расстояние_от_начала маршрута	DECIMAL(6,2)	NULL	
	Arrival_Time	Время_прибытия	TIME	NULL	

Рис. 9. Таблица «Маршрут-остановка»

Первичный ключ **id_Route_Stop** имеет тип **INT** и ограничение **NOT NULL**, так как используется для уникальной идентификации записей и обеспечивает эффективную обработку числовых данных. Внешние ключи **id_Route** и **id_Stop** имеют типы, соответствующие первичным ключам связанных таблиц, и обеспечивают ссылочную целостность данных. Атрибут **Stop_Order** имеет тип **INT** и ограничение **NOT NULL**, поскольку определяет порядок следования остановок и поддерживает операции сортировки. Атрибут **Distance_From_Start** имеет тип **DECIMAL(6,2)**, так как хранит числовое значение с дробной частью, обеспечивая необходимую точность. Атрибут **Arrival_Time** имеет тип **TIME** и может быть необязательным. Использование числовых, символьных и временных типов данных соответствует их классификации в СУБД :contentReference[oaicite:0]index=0.

Источник: <https://dev.mysql.com/doc/refman/8.0/en/create-table.html>

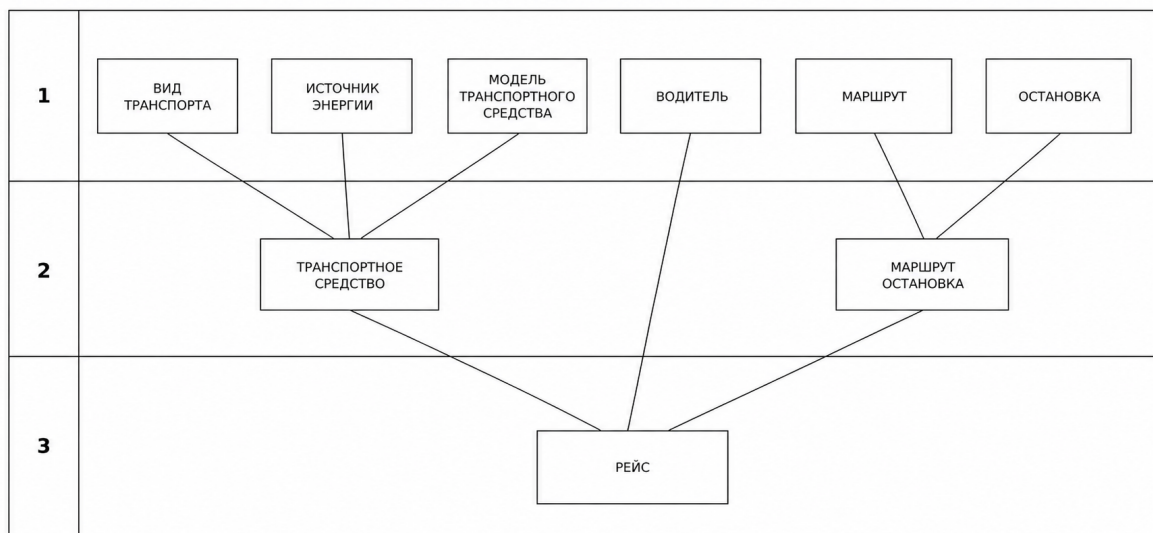


Рис. 10: Уровни заполнения базы данных

3.3 Схема базы данных на русском языке

На рисунке 11 представлена схема базы данных на русском языке.
На рисунке 12 представлена схема базы данных на английском языке.

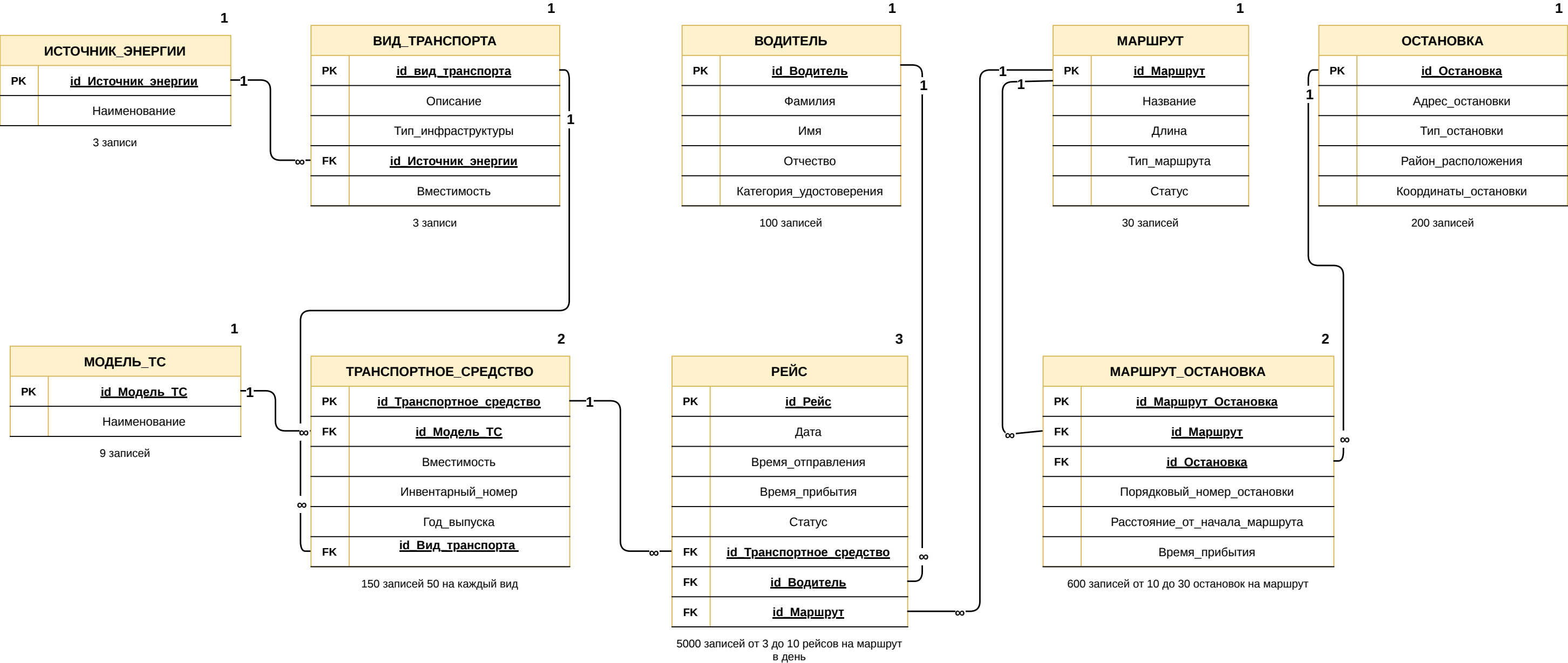


Рисунок 11 — Схема базы данных на русском языке

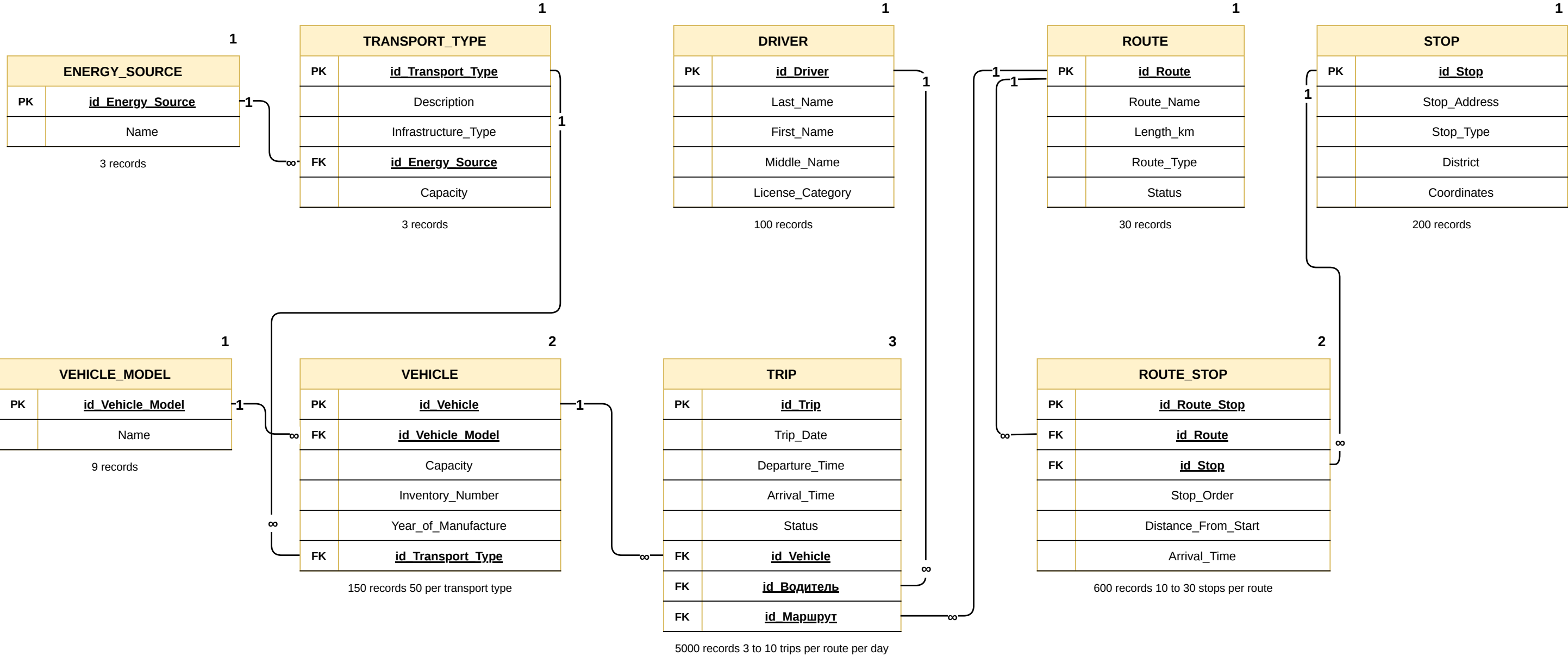


Рисунок 12 — Схема базы данных на английском языке

3.4 Генерация данных

Для генерации записей в базу данных был написан скрипт на языке программирования Python, представленный в приложении Б. Скрипт использует исключительно стандартную библиотеку Python (модули `random`, `datetime`, `pathlib`, `argparse`, `subprocess`) и не требует сторонних пакетов. Справочные таблицы (`ENERGY_SOURCE`, `VEHICLE_MODEL`, `TRANSPORT_TYPE`, `ROUTE`, `STOP`) заполняются фиксированными данными из программных списков. Основные таблицы (`DRIVER`, `VEHICLE`, `ROUTE_STOP`, `TRIP`) заполняются псевдослучайными данными, сгенерированными с помощью модуля `random` с фиксированным зерном (`seed`). Скрипт формирует файл `generated_data.sql`, содержащий команды `TRUNCATE TABLE ... RESTART IDENTITY CASCADE` для очистки и `INSERT INTO` для заполнения таблиц, после чего при наличии флага `-execute` передаёт файл утилите `psql` для выполнения.

4 Программирование

4.1 Запрос 1

Найти водителей, которые выполняли рейсы на транспортном средстве А (инвентарный номер INV-08-989) и проезжали остановку Б (Stop 157).

Исходный код запроса на языке SQL представлен в листинге 1.

```
1      SELECT DISTINCT
2          d.id_driver ,
3          d.last_name ,
4          d.first_name ,
5          d.middle_name ,
6          d.license_category ,
7          v.inventory_number AS vehicle ,
8          s.stop_name AS stop
9      FROM vehicle v
10     JOIN trip t
11     ON t.id_vehicle = v.id_vehicle
12     JOIN driver d
13     ON d.id_driver = t.id_driver
14     JOIN route_stop rs
15     ON rs.id_route = t.id_route
16     JOIN stop s
17     ON s.id_stop = rs.id_stop
18     WHERE v.inventory_number = 'INV-08-989'
19     AND s.stop_name = 'Stop 157'
20     ORDER BY
21         d.last_name ,
22         d.first_name ,
23         d.id_driver;
```

Листинг 1: Текст запроса 1 на SQL

Результат выполнения запроса 1 представлен на рис. 13.

```
PS C:\Users\shami\OneDrive\Documents\dataBase\myReport\code\python> psql -U postgres -d urban_transport_db -f query1.sql
Пароль пользователя postgres:
 id_driver | last_name | first_name | middle_name | license_category | vehicle_a | stop_b
-----
    1316 | Smirnova | Tatiana   | Ivanovna   | D1               | INV-08-989 | Stop 157
      99 | Titova   | Ekaterina | Alexeevna  | D2               | INV-08-989 | Stop 157
   1819 | Titova   | Ekaterina | Alexeevna  | D                | INV-08-989 | Stop 157
(3 строки)
```

Рис. 13: Результат выполнения запроса 1

Время выполнения запроса: менее 1 мс.

План выполнения запроса 1 представлен на рис. 14. В плане используются индексные обращения к таблицам vehicle, trip, route_stop,

stop и driver. Соединение таблиц выполняется через несколько операций Nested Loop Inner Join, после чего результат сортируется и из него удаляются повторяющиеся строки с помощью операции Unique.

Graphical Analysis Statistics			Rows
#	Node	Plan	
1.	→ Unique (cost=26.41..26.42 rows=1 width=50)		1
2.	→ Sort (cost=26.41..26.41 rows=1 width=50)		1
3.	→ Nested Loop Inner Join (cost=5.42..26.4 rows=1 width=50)		1
4.	→ Nested Loop Inner Join (cost=5.15..26.1 rows=1 width=23)		1
5.	→ Nested Loop Inner Join (cost=5..21.71 rows=24 width=19)		24
6.	→ Nested Loop Inner Join (cost=4.72..20.49 rows=2 width=19)		2
7.	→ Index Scan using vehicle_inventory_number_key on public.vehicle as v (cost=0.41..8.43 rows=... Index Cond: ((v.inventory_number)::text = 'INV-08-989':text))		1
8.	→ Bitmap Heap Scan on public.trip as t (cost=4.31..12.04 rows=2 width=12) Recheck Cond: (v.id_vehicle = t.id_vehicle)		2
9.	→ Bitmap Index Scan using idx_trip_vehicle (cost=0..4.31 rows=2 width=0) Index Cond: (t.id_vehicle = v.id_vehicle)		2
10.	→ Index Scan using idx_route_stop_route on public.route_stop as rs (cost=0.28..0.49 rows=12 width=8) Index Cond: (rs.id_route = t.id_route)		12
11.	→ Index Scan using stop_pkey on public.stop as s (cost=0.14..0.18 rows=1 width=12) Filter: ((s.stop_name)::text = 'Stop 157':text) Index Cond: (s.id_stop = rs.id_stop)		1
12.	→ Index Scan using driver_pkey on public.driver as d (cost=0.28..0.3 rows=1 width=31) Index Cond: (d.id_driver = t.id_driver)		1

Рис. 14: План выполнения запроса 1

Графическое представление последовательности выполнения запроса 1 представлено на рис. 15.

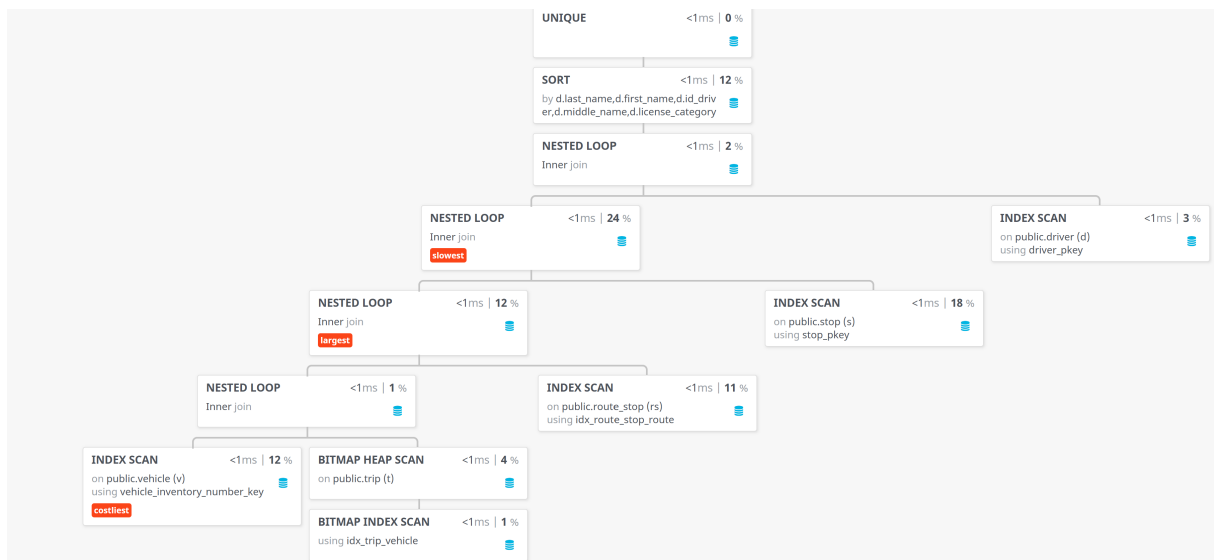


Рис. 15: Графическое представление последовательности выполнения запроса 1

Реляционная формула выполнения запроса:

$$\tau(id_driver, last_name, license_category)(DISTINCT \\ \sigma_{inventory_number='INV-08-989' \wedge stop_name='Stop 157'} \\ (vehicle \bowtie trip \bowtie driver \bowtie route_stop \bowtie stop))$$

Текстовое объяснение последовательности выполнения запроса.

Запрос начинается с таблицы **vehicle** и последовательно соединяет **trip** для получения всех рейсов этого ТС, **driver** для данных водителя, **route_stop** для остановок маршрута и **stop** для названия остановки. Условие **WHERE** одновременно фильтрует по инвентарному номеру ТС (INV-08-989) и по названию остановки (Stop 157). Ключевое слово **DISTINCT** исключает дублирующиеся строки: один водитель мог проехать заданную остановку на данном ТС несколько раз. В базе данных найдено **3 водителя**.

4.2 Запрос 2

Посчитать число маршрутов, по которым выполняли рейсы транспортные средства с источником энергии A (Diesel).

Исходный код запроса на языке SQL представлен в листинге 2.

```
1      SELECT
2      es.id_energy_source ,
3      es.name AS energy_source ,
4      COUNT(DISTINCT t.id_route) AS route_count
5  FROM energy_source es
6  JOIN transport_type tt
7  ON tt.id_energy_source = es.id_energy_source
8  JOIN vehicle v
9  ON v.id_transport_type = tt.id_transport_type
10 JOIN trip t
11 ON t.id_vehicle = v.id_vehicle
12 WHERE es.name = 'Diesel'
13 GROUP BY
14 es.id_energy_source ,
15 es.name ;
```

Листинг 2: Текст запроса 2 на SQL

Результат выполнения запроса 2 представлен на рис. 16.

```
PS C:\Users\shami\OneDrive\Documents\dataBase\myReport\code\python> psql -U postgres -d urban_transport_db -f query2.sql
Пароль пользователя postgres:

 id_energy_source | energy_source_a | route_count
-----
1 | Diesel          |          55
(1 строка)
```

Рис. 16: Результат выполнения запроса 2

Время выполнения запроса: около 132 мс.

План выполнения запроса 2 представлен на рис. 17. Сначала выполняется поиск источника энергии Diesel по индексу `energy_source_name_key`. Затем таблица `transport_type` соединяется с найденным источником энергии, после чего через индексные обращения к таблицам `vehicle` и `trip` выбираются связанные транспортные средства и рейсы. В конце выполняется сортировка по маршрутам и агрегирование для подсчёта количества различных маршрутов.

Graphical Analysis Statistics			Rows
#	Node	Plan	
1.	→ Aggregate (cost=364.9..368.66 rows=1 width=130)		1
2.	→ Sort (cost=364.9..366.77 rows=751 width=126)		751
3.	→ Nested Loop Inner Join (cost=8.76..329.03 rows=751 width=126)		751
4.	→ Nested Loop Inner Join (cost=8.47..118.69 rows=497 width=126)		497
5.	→ Hash Inner Join (cost=8.18..19.7 rows=1 width=126) Hash Cond: (tt.id_energy_source = es.id_energy_source)		1
6.	→ Seq Scan on public.transport_type as tt (cost=0..11.2 rows=120 width=8)		120
7.	→ Hash (cost=8.17..8.17 rows=1 width=122)		1
8.	→ Index Scan using energy_source_name_key on public.energy_source as es (cost=0.15..8.17 ro... Index Cond: ((es.name)::text = 'Diesel')::text)		1
9.	→ Index Scan using idx_vehicle_type on public.vehicle as v (cost=0.29..69.17 rows=2982 width=8) Index Cond: (v.id_transport_type = tt.id_transport_type)		2982
10.	→ Index Scan using idx_trip_vehicle on public.trip as t (cost=0.29..0.4 rows=2 width=8) Index Cond: (t.id_vehicle = v.id_vehicle)		2

Рис. 17: План выполнения запроса 2

Графическое представление последовательности выполнения запроса 2 представлено на рис. 18.

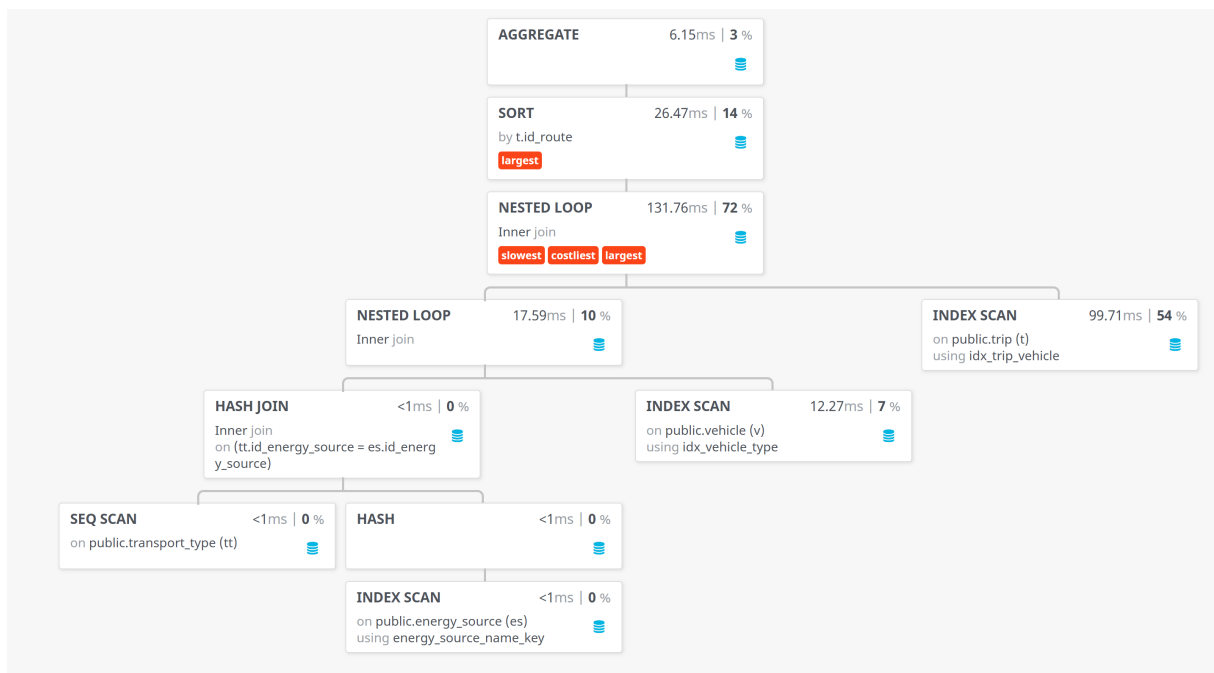


Рис. 18: Графическое представление последовательности выполнения запроса 2

Реляционная формула выполнения запроса:

$$\begin{aligned}
 &\tau(\text{energy_source}, \text{route_count}) (\\
 &\quad \gamma(\text{id_energy_source}, \text{COUNT}(\text{DISTINCT id_route}) \rightarrow \text{route_count}) \\
 &\quad (\sigma_{\text{name}='Diesel'}(\text{energy_source}) \bowtie \text{transport_type} \bowtie \text{vehicle} \bowtie \text{trip}))
 \end{aligned}$$

Текстовое объяснение последовательности выполнения запроса.

Запрос начинается с таблицы `energy_source` и фильтрует по названию через `WHERE es.name = 'Diesel'`. Затем через цепочку `JOIN` последовательно подключаются `transport_type`, `vehicle` и `trip`. Использование обычных `JOIN` (без `LEFT`) означает, что в результат включаются только строки, для которых существуют связанные рейсы.

Агрегатная функция `COUNT(DISTINCT t.id_route)` подсчитывает количество уникальных маршрутов. Группировка производится по идентификатору и названию источника энергии.

Результатом является одна строка: источник энергии **Diesel** обслуживал **55** маршрутов.

4.3 Запрос 3

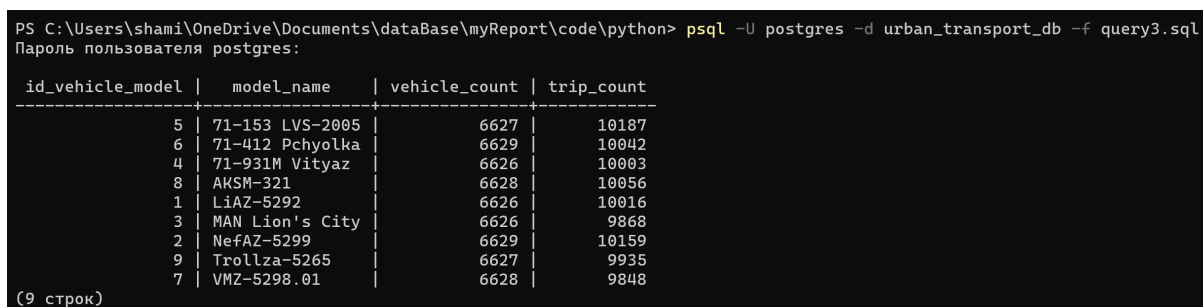
Для всех моделей транспортных средств посчитать число транспортных средств и число рейсов, построить комбинированную гистограмму.

Исходный код запроса на языке SQL представлен в листинге 3.

```
1      SELECT
2      vm.id_vehicle_model ,
3      vm.name                               AS model_name ,
4      COUNT(DISTINCT v.id_vehicle)         AS vehicle_count ,
5      COUNT(t.id_trip)                     AS trip_count
6      FROM vehicle_model vm
7      LEFT JOIN vehicle v ON v.id_vehicle_model = vm.
           id_vehicle_model
8      LEFT JOIN trip t   ON t.id_vehicle      = v.
           id_vehicle
9      GROUP BY vm.id_vehicle_model , vm.name
10     ORDER BY vm.name ;
```

Листинг 3: Текст запроса 3 на SQL

Результат выполнения запроса 3 представлен на рис. 19.



id_vehicle_model	model_name	vehicle_count	trip_count
5	71-153 LVS-2005	6627	10187
6	71-412 Pchyolka	6629	10042
4	71-931M Vityaz	6626	10003
8	AKSM-321	6628	10056
1	LiAZ-5292	6626	10016
3	MAN Lion's City	6626	9868
2	NefAZ-5299	6629	10159
9	Trollza-5265	6627	9935
7	VMZ-5298.01	6628	9848

Рис. 19: Результат выполнения запроса 3

Время выполнения запроса: около 104 мс.

План выполнения запроса 3 представлен на рис. 20. В плане выполняются последовательные чтения таблиц `vehicle_model`, `vehicle` и `trip`. Затем данные о транспортных средствах и рейсах соединяются с помощью `Hash Right Join`, после чего выполняется сортировка по модели транспортного средства. Далее результат соединяется с таблицей моделей через `Merge Left Join`, агрегируется по идентификатору модели и сортируется по названию модели.

Graphical Analysis Statistics			Rows
#	Node	Plan	
1.	→ Sort (cost=18049.44..18050.24 rows=320 width=238)		320
2.	→ Aggregate (cost=11220.09..18036.12 rows=320 width=238)		320
3.	→ Incremental Sort (cost=11220.09..17357.07 rows=90114 width=230)		90114
4.	→ Merge Left Join (cost=11204.4..12557.71 rows=90114 width=230)		90114
5.	→ Sort (cost=26.52..27.32 rows=320 width=222)		320
6.	→ Seq Scan on public.vehicle_model as vm (cost=0..13.2 rows=320 width=222)		320
7.	→ Sort (cost=11177.89..11403.17 rows=90114 width=12)		90114
8.	→ Hash Right Join (cost=1781.04..3761.75 rows=90114 width=12) Hash Cond: (t.id_vehicle = v.id_vehicle)		90114
9.	→ Seq Scan on public.trip as t (cost=0..1744.14 rows=90114 width=8)		90114
10.	→ Hash (cost=1035.46..1035.46 rows=59646 width=8)		59646
11.	→ Seq Scan on public.vehicle as v (cost=0..1035.46 rows=59646 width=8)		59646

Рис. 20: План выполнения запроса 3

Графическое представление последовательности выполнения запроса 3 представлено на рис. 21.

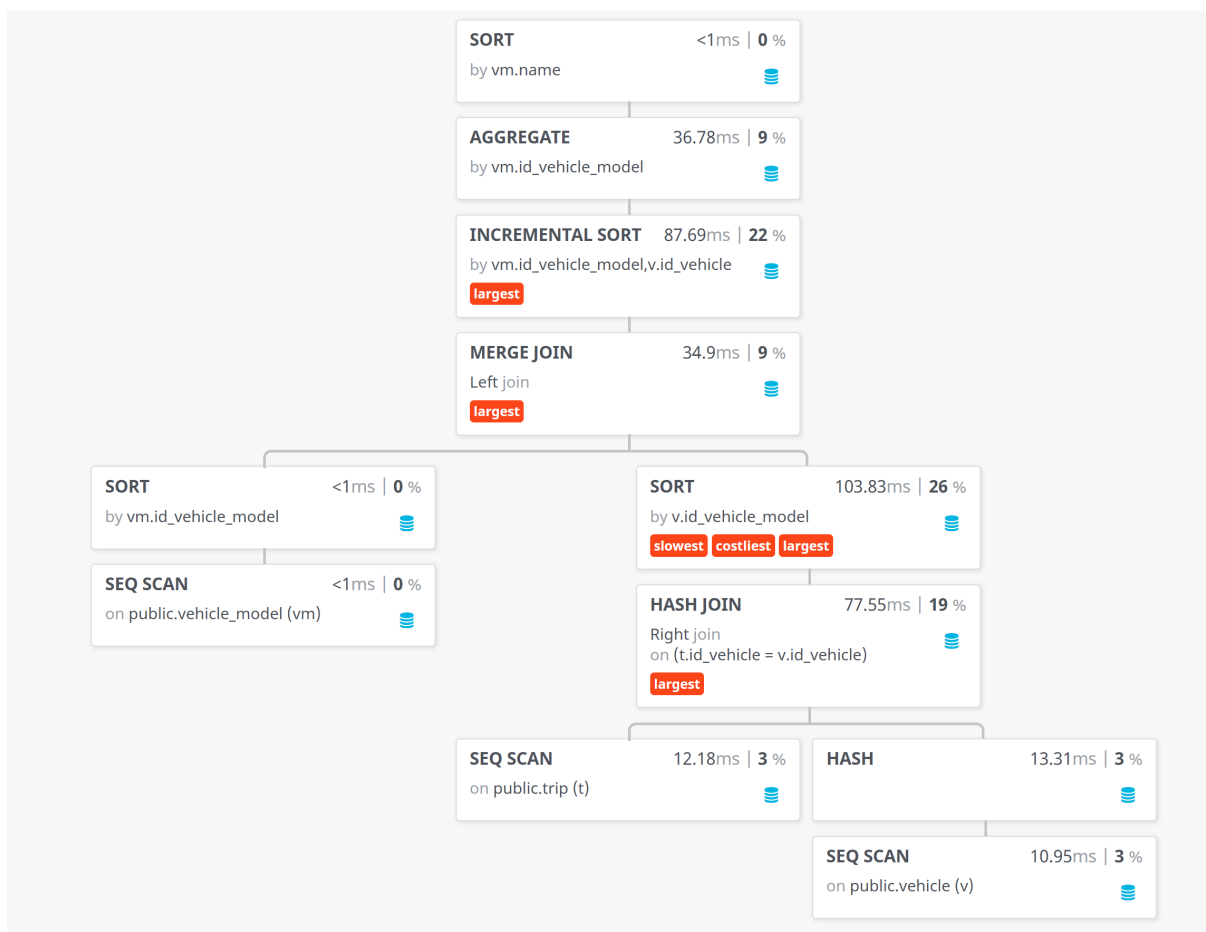


Рис. 21: Графическое представление последовательности выполнения запроса 3

Реляционная формула выполнения запроса:

$$\tau(model_name, vehicle_count, trip_count)(\gamma(id_vehicle_model, COUNT(id_vehicle) \rightarrow vehicle_count, COUNT(id_trip) (vehicle_model \bowtie vehicle \bowtie trip)))$$

Текстовое объяснение последовательности выполнения запроса.

Запрос начинается с таблицы `vehicle_model`. Через LEFT JOIN присоединяется `vehicle` (включая модели без ТС), затем через LEFT JOIN — `trip`. Результат группируется по идентификатору и названию модели. Вычисляются два агрегата: `COUNT(DISTINCT id_vehicle)` — количество уникальных ТС, `COUNT(id_trip)` — общее число рейсов. В базе данных содержится **9 моделей**, рейсов от 9848 до 10187.

Комбинированная гистограмма, построенная по результатам запроса, представлена на рис. 22. На диаграмме синим цветом показано количество транспортных средств по моделям по левой оси, оранжевым цветом — количество рейсов по тем же моделям по правой оси.

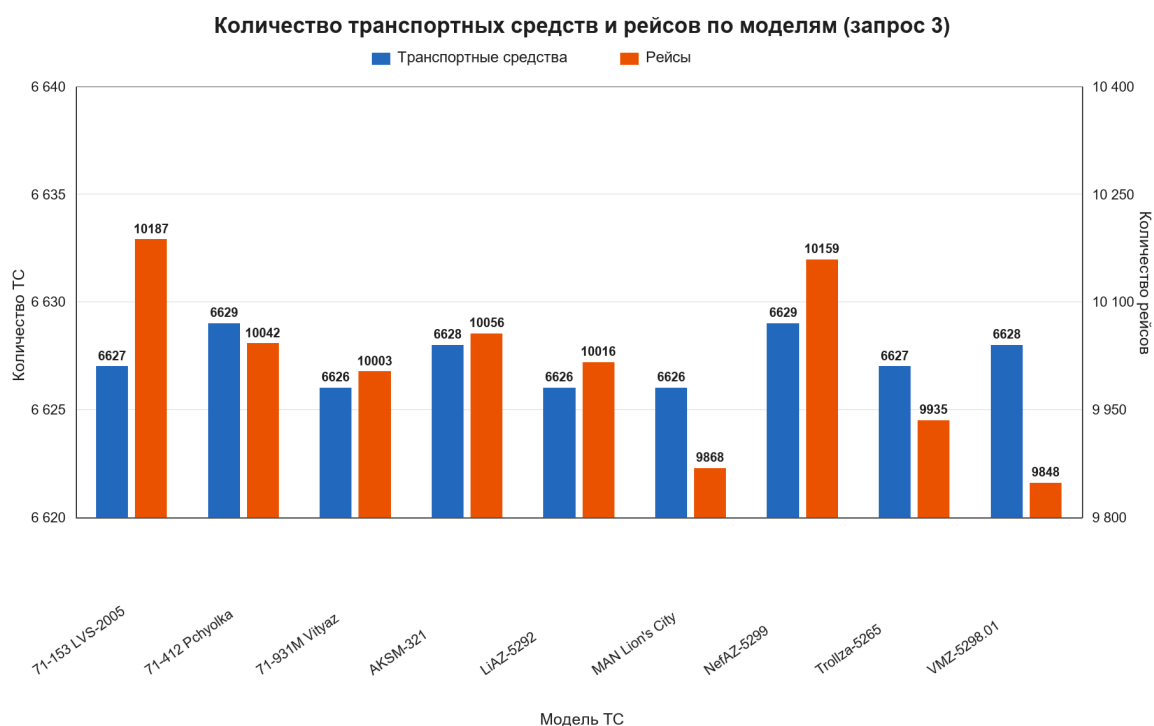


Рис. 22: Комбинированная гистограмма — количество транспортных средств и рейсов по моделям (запрос 3)

4.4 Запрос 4

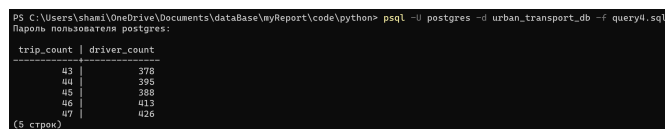
Посчитать число водителей с равным числом рейсов, построить гистограмму.

Исходный код запроса на языке SQL представлен в листинге 4.

```
1      SELECT
2      driver_trip_counts.trip_count ,
3      COUNT(*) AS driver_count
4      FROM (
5      SELECT
6      d.id_driver ,
7      COUNT(t.id_trip) AS trip_count
8      FROM driver d
9      LEFT JOIN trip t
10     ON t.id_driver = d.id_driver
11     GROUP BY
12     d.id_driver
13     ) AS driver_trip_counts
14     GROUP BY
15     driver_trip_counts.trip_count
16     ORDER BY
17     driver_trip_counts.trip_count;
```

Листинг 4: Текст запроса 4 на SQL

Результат выполнения запроса 4 представлен на рис. 23.



trip_count	driver_count
43	378
44	395
45	388
46	413
47	426

Рис. 23: Результат выполнения запроса 4

Время выполнения запроса: менее 1 мс.

План выполнения запроса 4 представлен на рис. 24. Сначала выполняются последовательные чтения таблиц `driver` и `trip`. Затем данные соединяются по идентификатору водителя с помощью `Hash Right Join`. После этого внутренний уровень запроса группирует данные по водителю и подсчитывает число рейсов каждого водителя. Внешний уровень выполняет повторное агрегирование по количеству рейсов, а затем сортирует результат по возрастанию значения `trip_count`.

Graphical Analysis Statistics			Rows
#	Node	Plan	
1.	→ Sort (cost=2552.4..2552.9 rows=200 width=16)		200
2.	→ Aggregate (cost=2542.76..2544.76 rows=200 width=16)		200
3.	→ Aggregate (cost=2492.76..2512.76 rows=2000 width=12)		2000
4.	→ Hash Right Join (cost=61..2042.19 rows=90114 width=8) Hash Cond: (t.id_driver = d.id_driver)		90114
5.	→ Seq Scan on public.trip as t (cost=0..1744.14 rows=90114 width=8)		90114
6.	→ Hash (cost=36..36 rows=2000 width=4)		2000
7.	→ Seq Scan on public.driver as d (cost=0..36 rows=2000 width=4)		2000

Рис. 24: План выполнения запроса 4

Графическое представление последовательности выполнения запроса 4 представлено на рис. 25.

Графическое представление последовательности выполнения запроса 4

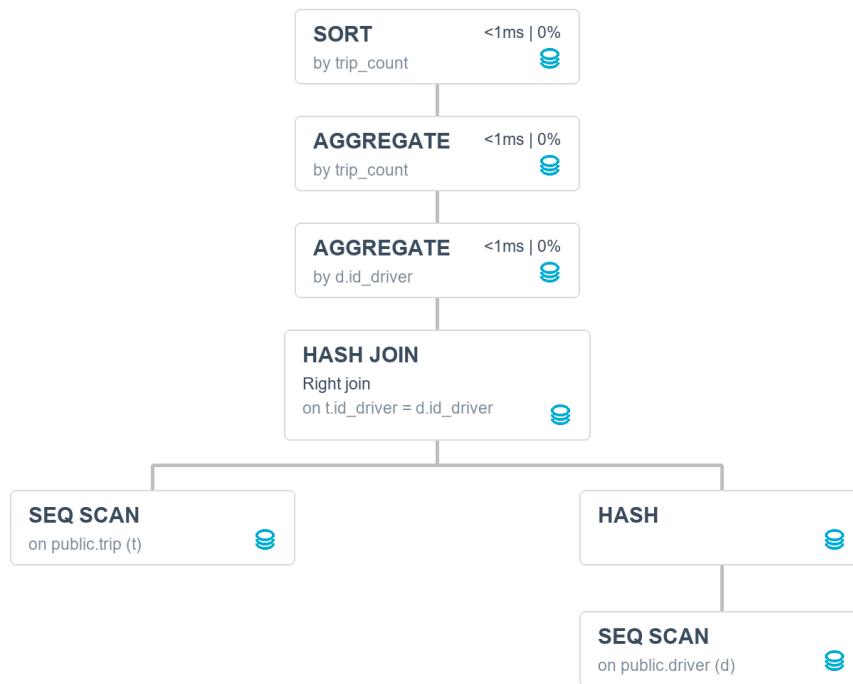


Рис. 25: Графическое представление последовательности выполнения запроса 4

Реляционная формула выполнения запроса:

$$\begin{aligned}
 &\tau(trip_count, driver_count)(\\
 &\quad \gamma(trip_count, COUNT(*) \rightarrow driver_count)(\\
 &\quad \quad \gamma(id_driver, COUNT(id_trip) \rightarrow trip_count)(driver \bowtie trip)))
 \end{aligned}$$

Текстовое объяснение последовательности выполнения запроса.

Подзапрос `driver_trip_counts` соединяет `driver` и `trip` через `LEFT JOIN`, группирует по водителю и подсчитывает количество рейсов. `LEFT JOIN` обеспечивает включение водителей без рейсов (значение 0).

Внешний запрос группирует строки результата подзапроса по значению `trip_count` и с помощью `COUNT(*)` подсчитывает, сколько водителей попало в каждую группу. Результат сортируется по возрастанию числа рейсов.

Итоговая таблица показывает распределение водителей по числу выполненных рейсов: от **43** до **47** рейсов, наибольшая группа — 47 рейсов (**426 водителей**).

Гистограмма распределения водителей по числу рейсов представлена на рис. 26.



Рис. 26: Гистограмма — распределение водителей по числу рейсов (запрос 4)

4.5 Запрос 5

Найти остановки с наибольшим и наименьшим числом маршрутов.
Исходный код запроса на языке SQL представлен в листинге 5.

```
1      SELECT
2          'maximum' AS result_type ,
3          s.id_stop ,
4          s.stop_name ,
5          s.stop_address ,
6          COUNT(DISTINCT rs.id_route) AS route_count
7      FROM stop s
8      LEFT JOIN route_stop rs
9      ON rs.id_stop = s.id_stop
10     GROUP BY
11         s.id_stop ,
12         s.stop_name ,
13         s.stop_address
14     HAVING COUNT(DISTINCT rs.id_route) = (
15         SELECT MAX(stop_counts.route_count)
16     FROM (
17         SELECT COUNT(DISTINCT rs2.id_route) AS route_count
18     FROM stop s2
19     LEFT JOIN route_stop rs2
20     ON rs2.id_stop = s2.id_stop
21     GROUP BY
22         s2.id_stop
23     ) AS stop_counts
24     )
25
26     UNION ALL
27
28     SELECT
29         'minimum' AS result_type ,
30         s.id_stop ,
31         s.stop_name ,
32         s.stop_address ,
33         COUNT(DISTINCT rs.id_route) AS route_count
34     FROM stop s
35     LEFT JOIN route_stop rs
36     ON rs.id_stop = s.id_stop
37     GROUP BY
38         s.id_stop ,
39         s.stop_name ,
40         s.stop_address
41     HAVING COUNT(DISTINCT rs.id_route) = (
42         SELECT MIN(stop_counts.route_count)
43     FROM (
44         SELECT COUNT(DISTINCT rs2.id_route) AS route_count
```

```

45      FROM stop s2
46      LEFT JOIN route_stop rs2
47      ON rs2.id_stop = s2.id_stop
48      GROUP BY
49      s2.id_stop
50      ) AS stop_counts
51      )
52      ORDER BY
53      result_type ,
54      stop_name ;

```

Листинг 5: Текст запроса 5 на SQL

Результат выполнения запроса 5 представлен на рис. 27.

```

PS C:\Users\shami\OneDrive\Documents\dataBase\myReport\code\python> psql -U postgres -d urban_transport_db -f query5.sql
Пароль пользователя postgres:

```

result_type	id_stop	stop_name	stop_address	route_count
maximum	110	Stop 110	Street 51, No. 11	14
maximum	27	Stop 27	Street 28, No. 3	14
maximum	81	Stop 81	Street 22, No. 7	14
minimum	100	Stop 100	Street 41, No. 1	1
minimum	105	Stop 105	Street 46, No. 6	1
minimum	153	Stop 153	Street 34, No. 4	1
minimum	98	Stop 98	Street 39, No. 24	1

(7 строк)

Рис. 27: Результат выполнения запроса 5

Время выполнения запроса: менее 1 мс.

План выполнения запроса 5 представлен на рис. 28. Запрос состоит из двух ветвей, объединенных оператором UNION ALL: первая ветвь отбирает остановки с максимальным числом маршрутов, вторая ветвь - с минимальным. Для каждой ветви выполняется соединение таблиц `stop` и `route_stop`, затем сортировка и агрегация по остановкам с подсчетом уникальных маршрутов. Вложенные подзапросы отдельно вычисляют значения MAX и MIN, после чего внешний фильтр HAVING оставляет только подходящие остановки. В конце результаты двух ветвей объединяются и сортируются по типу результата и названию остановки.

Graphical	Analysis	Statistics	
#	Node	Rows	Plan
1.	→ Sort (cost=214.34..214.35 rows=1 width=69)	1	
2.	→ Aggregate (cost=108.16..108.17 rows=1 width=8)	1	
3.	→ Aggregate (cost=94.51..105.66 rows=200 width=12)	200	
4.	→ Sort (cost=94.51..97.56 rows=1220 width=8)	1220	
5.	→ Hash Right Join (cost=7.5..31.97 rows=1220 width=8) Hash Cond: (rs2.id_stop = s2.id_stop)	1220	
6.	→ Seq Scan on public.route_stop as rs2 (cost=0..21.2 rows=1220 width=8)	1220	
7.	→ Hash (cost=5..5 rows=200 width=4)	200	
8.	→ Seq Scan on public.stop as s2 (cost=0..5 rows=200 width=4)	200	
9.	→ Aggregate (cost=94.51..106.16 rows=1 width=69) Filter: (count(DISTINCT rs.id_route) = (InitPlan 1).col1)	1	
10.	→ Sort (cost=94.51..97.56 rows=1220 width=33)	1220	
11.	→ Hash Right Join (cost=7.5..31.97 rows=1220 width=33) Hash Cond: (rs.id_stop = s.id_stop)	1220	
12.	→ Seq Scan on public.route_stop as rs (cost=0..21.2 rows=1220 width=8)	1220	
13.	→ Hash (cost=5..5 rows=200 width=29)	200	
14.	→ Seq Scan on public.stop as s (cost=0..5 rows=200 width=29)	200	

Рис. 28: План выполнения запроса 5

Графическое представление последовательности выполнения запроса 5 представлено на рис. 29. На диаграмме видно, что после начальной сортировки выполняется узел **Append**, который объединяет результаты двух одинаковых по структуре ветвей. В каждой ветви сначала строится агрегированный набор количества маршрутов по остановкам, затем он сравнивается с рассчитанным максимумом или минимумом. Основными операциями плана являются последовательное сканирование таблиц, хеш-соединение, сортировка и агрегация.

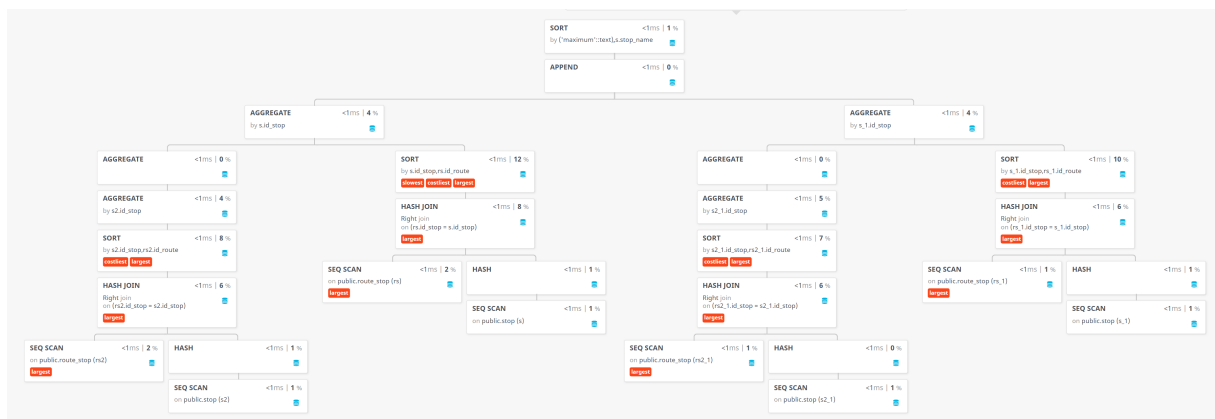


Рис. 29: Графическое представление последовательности выполнения запроса 5

Реляционная формула выполнения запроса:

$$\tau(result_type, id_stop, route_count) \left(\begin{aligned} &\sigma_{route_count=\max}(\gamma(id_stop, COUNT(id_route) \rightarrow rc)(stop \bowtie route_stop)) \\ \cup \\ &\sigma_{route_count=\min}(\gamma(id_stop, COUNT(id_route) \rightarrow rc)(stop \bowtie route_stop)) \end{aligned} \right)$$

Текстовое объяснение последовательности выполнения запроса.

Каждая из двух частей UNION ALL выполняет одинаковую агрегацию: соединяет **stop** и **route_stop** через LEFT JOIN и для каждой остановки подсчитывает количество уникальных маршрутов. Условие HAVING сравнивает полученное значение с глобальным максимумом или минимумом, вычисляемым во вложенном подзапросе. Первая часть отбирает остановки с максимальным числом маршрутов, вторая — с минимальным. Результаты объединяются операцией UNION ALL.

Результат содержит **3 остановки** с максимальным числом маршрутов (14 каждая) и **4 остановки** с минимальным (по 1 маршруту).

4.6 Запрос 6

Найти транспортные средства, которые выполнили меньше рейсов, чем транспортное средство С

Исходный код запроса на языке SQL представлен в листинге 6.

```
1      SELECT
2          v.id_vehicle ,
3          v.inventory_number ,
4          vm.name          AS model_name ,
5          COUNT(t.id_trip)  AS trip_count
6  FROM  vehicle v
7  JOIN  vehicle_model vm
8      ON v.id_vehicle_model = vm.id_vehicle_model
9  JOIN  trip t
10     ON v.id_vehicle = t.id_vehicle
11  GROUP BY
12      v.id_vehicle ,
13      v.inventory_number ,
14      vm.name
15  HAVING COUNT(t.id_trip) < (
16      SELECT COUNT(*)
17      FROM trip
18      WHERE id_vehicle = 4
19  )
20  ORDER BY trip_count DESC;
```

Листинг 6: Текст запроса 6 на SQL

Результат выполнения запроса 6 представлен на рис. 30.

1	id_vehicle	inventory_number	model_name	trip_count
2	-----+-----+-----+-----			
3	11632	INV-04-2751	Trollza-5265	2
4	33090	INV-12-200	71-931M Vityaz	2
5	6029	INV-03-122	VMZ-5298.01	2
6	20011	INV-07-2031	MAN Lion's City	2
7	34895	INV-12-2005	Trollza-5265	2
8	11636	INV-04-2755	71-931M Vityaz	2
9	34897	INV-12-2007	NefAZ-5299	2
10	34898	INV-12-2008	MAN Lion's City	2
11	33091	INV-12-201	71-153 LVS-2005	2
12	20018	INV-07-2038	LiAZ-5292	2
13	11637	INV-04-2756	71-153 LVS-2005	2
14	20022	INV-07-2042	71-153 LVS-2005	2
15	34906	INV-12-2016	NefAZ-5299	2
16	11638	INV-04-2757	71-412 Pchjolka	2
17	7127	INV-03-1220	VMZ-5298.01	2
18	33092	INV-12-202	71-412 Pchjolka	2
19	18185	INV-07-205	71-931M Vityaz	2
20	11641	INV-04-2760	Trollza-5265	2
21	11642	INV-04-2761	LiAZ-5292	2
22	34915	INV-12-2025	NefAZ-5299	2
23	4354	INV-02-1355	71-412 Pchjolka	2
24	20037	INV-07-2057	NefAZ-5299	2

Рис. 30: Результат выполнения запроса 6

Время выполнения запроса: около 1,1 с.

План выполнения запроса 6 представлен на рис. 31. Сначала во вложенном подзапросе вычисляется количество рейсов транспортного средства с `id_vehicle = 4`. Для этого используется индексное сканирование таблицы `trip` по индексу `idx_trip_vehicle`, после чего выполняется агрегация `COUNT(*)`. Основная часть запроса соединяет таблицы `trip`, `vehicle` и `vehicle_model` с помощью хеш-соединений, группирует данные по транспортному средству и модели, затем сравнивает количество рейсов каждого транспортного средства со значением, полученным во вложенном подзапросе. На последнем этапе результат сортируется по убыванию `trip_count`.

Graphical Analysis Statistics			
#	Node	Rows	Plan
1.	→ Sort (cost=29361.18..29436.28 rows=30038 width=241)	30038	
2.	→ Aggregate (cost=4.33..4.34 rows=1 width=8)	1	
3.	→ Index Only Scan using idx_trip_vehicle on public.trip as trip (cost=0.29..4.33 rows=2 width=0) Index Cond: (trip.id_vehicle = 4)	2	
4.	→ Aggregate (cost=21602.27..23629.84 rows=30038 width=241) Filter: (count(t.id_trip) < (InitPlan 1).col1)	30038	
5.	→ Sort (cost=21602.27..21827.56 rows=90114 width=237)	90114	
6.	→ Hash Inner Join (cost=1798.24..4018.63 rows=90114 width=237) Hash Cond: (v.id_vehicle_model = vm.id_vehicle_model)	90114	
7.	→ Hash Inner Join (cost=1781.04..3761.75 rows=90114 width=23) Hash Cond: (t.id_vehicle = v.id_vehicle)	90114	
8.	→ Seq Scan on public.trip as t (cost=0..1744.14 rows=90114 width=8)	90114	
9.	→ Hash (cost=1035.46..1035.46 rows=59646 width=19)	59646	
10.	→ Seq Scan on public.vehicle as v (cost=0..1035.46 rows=59646 width=19)	59646	
11.	→ Hash (cost=13.2..13.2 rows=320 width=222)	320	
12.	→ Seq Scan on public.vehicle_model as vm (cost=0..13.2 rows=320 width=222)	320	

Рис. 31: План выполнения запроса 6

Графическое представление последовательности выполнения запроса 6 представлено на рис. 32. На диаграмме видно, что подзапрос для транспортного средства С выполняется отдельно и дает контрольное значение для условия HAVING. Основной поток плана строится из последовательного сканирования таблиц, хеширования справочных данных, хеш-соединений и сортировки перед агрегацией. Самой затратной частью является сортировка набора строк перед группировкой по транспортным средствам.

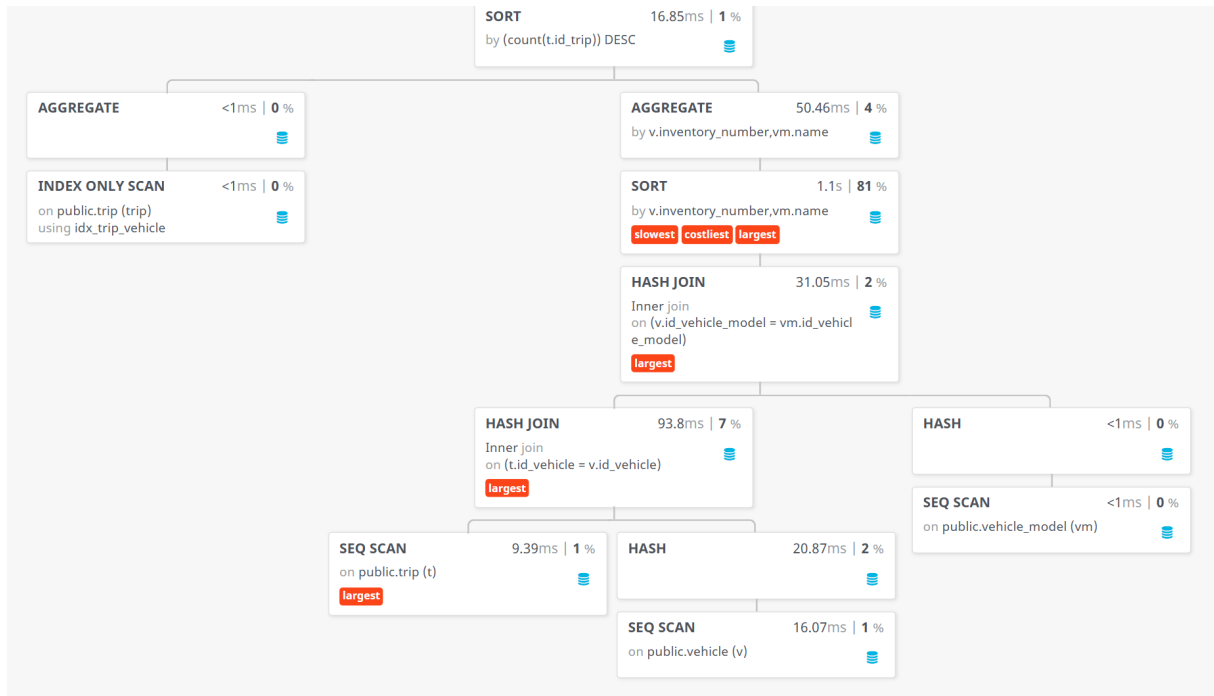


Рис. 32: Графическое представление последовательности выполнения запроса 6

Реляционная формула выполнения запроса:

$$\tau_{trip_count} DESC \left(\sigma_{trip_count < count_C} \left(\gamma_{id_vehicle, inventory_number, name; COUNT(*) \rightarrow trip_count} \left(vehicle \bowtie vehicle_model \bowtie trip \right) \right) \right),$$

$$\text{где } count_C = \gamma_{COUNT(*)} (\sigma_{id_vehicle=4}(trip))$$

Текстовое объяснение последовательности выполнения запроса.

Основной запрос соединяет таблицы `vehicle`, `vehicle_model` и `trip` через JOIN, группирует результат по транспортному средству и подсчитывает число рейсов каждого ТС с помощью `COUNT(t.id_trip)`.

Подзапрос в конструкции HAVING вычисляет число рейсов ТС `C` (`id_vehicle=4`). Условие HAVING оставляет только те ТС, у которых число рейсов строго меньше, чем у ТС `C`. Результат сортируется по убыванию числа рейсов.

ТС `C` (`INV-01-004`) выполнило **3 рейса**. Запрос вернул все транспортные средства, совершившие 1 или 2 рейса, — **35 007 транспортных средств**.

4.7 Запрос 7

Найти водителей, которые никогда не проезжали остановку А (Stop 157).

Исходный код запроса на языке SQL представлен в листинге 7.

```
1      SELECT
2      d.id_driver ,
3      d.last_name ,
4      d.first_name ,
5      d.middle_name ,
6      d.license_category
7  FROM driver d
8  WHERE NOT EXISTS (
9      SELECT 1
10     FROM trip t
11     JOIN route_stop rs
12     ON rs.id_route = t.id_route
13     JOIN stop s
14     ON s.id_stop = rs.id_stop
15     WHERE t.id_driver = d.id_driver
16     AND s.stop_name = 'Stop 157'
17 )
18 ORDER BY
19     d.last_name ,
20     d.first_name ,
21     d.id_driver;
```

Листинг 7: Текст запроса 7 на SQL

Результат выполнения запроса 7 представлен на рис. 33.

```
PS C:\Users\shami\OneDrive\Documents\dataBase\myReport\code\python> psql -U postgres -d urban_transport_db -f query7.sql
Пароль пользователя postgres:
```

id_driver	last_name	first_name	middle_name	license_category
255	Alekseev	Anna	Mikhailovna	D2
835	Alekseev	Anna	Mikhailovna	D
1055	Alekseev	Anna	Mikhailovna	D1
1495	Alekseev	Anna	Mikhailovna	D
1555	Alekseev	Anna	Mikhailovna	D
1715	Alekseev	Anna	Mikhailovna	D1
390	Andreev	Irina	Andreevna	D
554	Andreev	Irina	Andreevna	D1
1214	Andreev	Irina	Andreevna	D1
1434	Andreev	Irina	Andreevna	D2
1854	Andreev	Irina	Andreevna	D2
1994	Andreev	Irina	Andreevna	D1
93	Fedorov	Olga	Sergeevna	D2
113	Fedorov	Olga	Sergeevna	D1
793	Fedorov	Olga	Sergeevna	D
1113	Fedorov	Olga	Sergeevna	D2
1158	Guseva	Svetlana	Vladimirovna	D2
1061	Ivanov	Alexander	Alexandrovich	D1
227	Kozlov	Nikolay	Nikolaevich	D1
724	Kuznetsov	Andrey	Andreevich	D
186	Lebedev	Ivan	Ivanovich	D
1966	Lebedev	Ivan	Ivanovich	D
1752	Mikhailov	Elena	Dmitrievna	D2
309	Morozov	Alexey	Alexeevich	D2
1069	Morozov	Alexey	Alexeevich	D
1229	Morozov	Alexey	Alexeevich	D1
169	Orlova	Yulia	Pavlovna	D
1189	Orlova	Yulia	Pavlovna	D1
1989	Orlova	Yulia	Pavlovna	D2
542	Petrov	Dmitry	Dmitrievich	D1
982	Petrov	Dmitry	Dmitrievich	D1
1885	Popov	Mikhail	Mikhailovich	D
303	Sidorov	Sergey	Sergeevich	D2
563	Sidorov	Sergey	Sergeevich	D1
376	Smirnova	Tatiana	Ivanovna	D
696	Smirnova	Tatiana	Ivanovna	D2
836	Smirnova	Tatiana	Ivanovna	D1
1096	Smirnova	Tatiana	Ivanovna	D
1476	Smirnova	Tatiana	Ivanovna	D2
1051	Sokolov	Natalia	Alexandrovna	D
1119	Titova	Ekaterina	Alexeevna	D2
1639	Titova	Ekaterina	Alexeevna	D
98	Volkov	Pavel	Pavlovich	D2
378	Volkov	Pavel	Pavlovich	D

(еще строки)

Рис. 33: Результат выполнения запроса 7

Время выполнения запроса: около 11 мс.

План выполнения запроса 7 представлен на рис. 34. В запросе используется условие `NOT EXISTS`, которое в плане преобразуется в операцию `Hash Anti Join`. Сначала выполняется последовательное сканирование таблицы `driver`. Для правой части антисоединения строится набор водителей, которые проезжали остановку `Stop 157`: таблица `stop` фильтруется по названию остановки, затем соединяется с `route_stop`, а таблица `trip` просматривается по индексу `idx_trip_route`. После этого антисоединение оставляет только тех водителей, для которых в правой части не найдено совпадений. В конце результат сортируется по фамилии, имени и идентификатору водителя.

Graphical Analysis Statistics			
#	Node	Rows	Plan
1.	→ Sort (cost=316.21..316.22 rows=1 width=31)	1	
2.	→ Hash Anti Join (cost=267.7..316.2 rows=1 width=31) Hash Cond: (d.id_driver = t.id_driver)	1	
3.	→ Seq Scan on public.driver as d (cost=0..36 rows=2000 width=31)	2000	
4.	→ Hash (cost=198.99..198.99 rows=5497 width=4)	5497	
5.	→ Nested Loop Inner Join (cost=5.8..198.99 rows=5497 width=4)	5497	
6.	→ Hash Inner Join (cost=5.51..29.98 rows=6 width=4) Hash Cond: (rs.id_stop = s.id_stop)	6	
7.	→ Seq Scan on public.route_stop as rs (cost=0..21.2 rows=1220 width=8)	1220	
8.	→ Hash (cost=5.5..5.5 rows=1 width=4)	1	
9.	→ Seq Scan on public.stop as s (cost=0..5.5 rows=1 width=4) Filter ((s.stop_name)::text = 'Stop 157'::text)	1	
10.	→ Index Scan using idx_trip_route on public.trip as t (cost=0.29..19.16 rows=901 width=8) Index Cond: (t.id_route = rs.id_route)	901	

Рис. 34: План выполнения запроса 7

Графическое представление последовательности выполнения запроса 7 представлено на рис. 35. На диаграмме видно, что основная логика запроса выполняется через антисоединение: слева находится полный список водителей, справа - множество водителей, связанных с маршрутами, проходящими через остановку Stop 157. Самыми заметными операциями плана являются вложенный цикл для сопоставления рейсов с маршрутами, индексное сканирование таблицы `trip` и хеширование промежуточного набора для последующего Hash Anti Join.

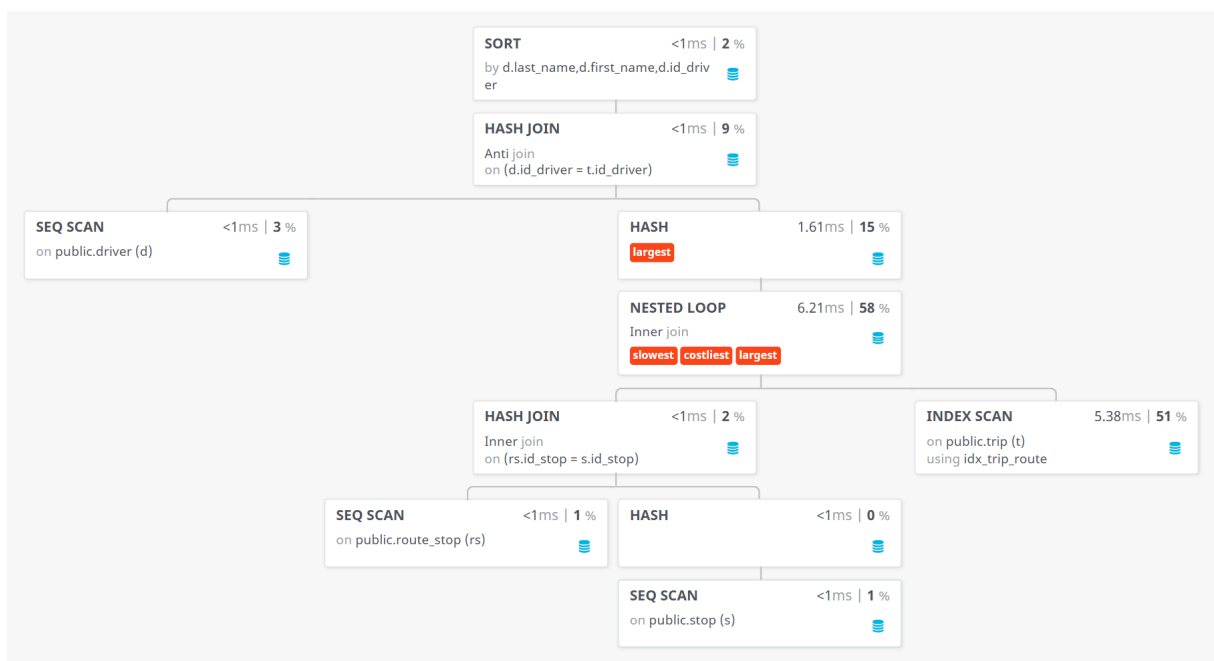


Рис. 35: Графическое представление последовательности выполнения запроса 7

Реляционная формула выполнения запроса:

$$\tau(id_driver, last_name, first_name)\left(\begin{aligned} &driver \setminus \tau(id_driver)\left(\right. \\ &\quad \sigma_{stop_name='Stop\ 157'}\left(\right. \\ &\quad\quad driver \bowtie trip \bowtie route_stop \bowtie stop \left. \right) \left. \right) \end{aligned} \right)$$

Текстовое объяснение последовательности выполнения запроса.

Для каждого водителя из таблицы **driver** выполняется коррелированный подзапрос NOT EXISTS. Подзапрос соединяет **trip**, **route_stop** и **stop**, фильтруя одновременно по текущему водителю (`t.id_driver = d.id_driver`) и по названию остановки (`s.stop_name = 'Stop 157'`).

Если подзапрос не возвращает ни одной строки (водитель ни разу не выполнял рейс по маршруту, включающему **Stop 157**), условие NOT EXISTS истинно и водитель включается в результат. Результат отсортирован по фамилии, имени и идентификатору.

4.8 Запрос 8

Для каждого источника энергии и модели транспортного средства посчитать число транспортных средств и построить 3D-гистограмму.

Исходный код запроса на языке SQL представлен в листинге 8.

```
1      SELECT
2      es.id_energy_source ,
3      es.name      AS energy_source ,
4      vm.id_vehicle_model ,
5      vm.name      AS model_name ,
6      COUNT(v.id_vehicle) AS vehicle_count
7  FROM energy_source es
8  CROSS JOIN vehicle_model vm
9  LEFT JOIN transport_type tt
10 ON tt.id_energy_source = es.id_energy_source
11 LEFT JOIN vehicle v
12 ON v.id_transport_type = tt.id_transport_type
13 AND v.id_vehicle_model = vm.id_vehicle_model
14 GROUP BY
15 es.id_energy_source , es.name ,
16 vm.id_vehicle_model , vm.name
17 ORDER BY es.name , vm.name ;
```

Листинг 8: Текст запроса 8 на SQL

Результат выполнения запроса 8 представлен на рис. 36.

```
PS C:\Users\shami\OneDrive\Documents\dataBase\myReport\code\python> psql -U postgres -d urban_transport_db -f query8.sql
Пароль пользователя postgres:
```

id_energy_source	energy_source	id_vehicle_model	model_name	vehicle_count
1	Diesel	5	71-153 LVS-2005	3692
1	Diesel	6	71-412 Pchyolka	3694
1	Diesel	4	71-931M Vityaz	3691
1	Diesel	8	AKSM-321	3694
1	Diesel	1	LiAZ-5292	3692
1	Diesel	3	MAN Lion's City	3693
1	Diesel	2	NefAZ-5299	3693
1	Diesel	9	Trollza-5265	3693
1	Diesel	7	VMZ-5298.01	3694
2	Electric overhead wire	5	71-153 LVS-2005	1642
2	Electric overhead wire	6	71-412 Pchyolka	1642
2	Electric overhead wire	4	71-931M Vityaz	1643
2	Electric overhead wire	8	AKSM-321	1642
2	Electric overhead wire	1	LiAZ-5292	1643
2	Electric overhead wire	3	MAN Lion's City	1641
2	Electric overhead wire	2	NefAZ-5299	1644
2	Electric overhead wire	9	Trollza-5265	1643
2	Electric overhead wire	7	VMZ-5298.01	1642
3	Electric rail current	5	71-153 LVS-2005	1293
3	Electric rail current	6	71-412 Pchyolka	1293
3	Electric rail current	4	71-931M Vityaz	1292
3	Electric rail current	8	AKSM-321	1292
3	Electric rail current	1	LiAZ-5292	1291
3	Electric rail current	3	MAN Lion's City	1292
3	Electric rail current	2	NefAZ-5299	1292
3	Electric rail current	9	Trollza-5265	1291
3	Electric rail current	7	VMZ-5298.01	1292

(27 строк)

Рис. 36: Результат выполнения запроса 8

Время выполнения запроса: около 110 мс.

План выполнения запроса 8 представлен на рис. 37. Сначала выполняются последовательные чтения таблиц `energy_source`, `transport_type`, `vehicle_model` и `vehicle`. Операция `CROSS JOIN` между источниками энергии и моделями реализуется через `Nested Loop Inner Join`. Затем таблица `transport_type` присоединяется к источникам энергии через `Hash Left Join`, а данные о транспортных средствах присоединяются через `Merge Left Join`. После соединений выполняется агрегирование по источнику энергии и модели, затем результат сортируется по названиям источника энергии и модели транспортного средства.

Graphical Analysis Statistics			
#	Node	Rows	Plan
1.	→ Sort (cost=175214.66..175646.66 rows=172800 width=352)	172800	
2.	→ Aggregate (cost=78587.92..104658.15 rows=172800 width=352)	172800	
3.	→ Incremental Sort (cost=78587.92..100917.1 rows=268407 width=348)	268407	
4.	→ Merge Left Join (cost=78528.46..84521.58 rows=268407 width=348)	268407	
5.	→ Sort (cost=72761.84..73193.84 rows=172800 width=348)	172800	
6.	→ Nested Loop Inner Join (cost=12.7..2205.32 rows=172800 width=348)	172800	
7.	→ Hash Left Join (cost=12.7..31.32 rows=540 width=126) Hash Cond: (es.id_energy_source = tt.id_energy_source)	540	
8.	→ Seq Scan on public.energy_source as es (cost=0..15.4 rows=540 width=122)	540	
9.	→ Hash (cost=11.2..11.2 rows=120 width=8)	120	
10.	→ Seq Scan on public.transport_type as tt (cost=0..11.2 rows=120 width=8)	120	
11.	→ Materialize (cost=0..14.8 rows=320 width=222)	320	
12.	→ Seq Scan on public.vehicle_model as vm (cost=0..13.2 rows=320 width=222)	320	
13.	→ Sort (cost=5766.62..5915.74 rows=59646 width=12)	59646	
14.	→ Seq Scan on public.vehicle as v (cost=0..1035.46 rows=59646 width=12)	59646	

Рис. 37: План выполнения запроса 8

Графическое представление последовательности выполнения запроса 8 представлено на рис. 38. На диаграмме видно, что основная стоимость запроса связана с сортировкой строк таблицы `vehicle`, последующим `Merge Join`, `Incremental Sort` и агрегированием. Это связано с тем, что запрос не содержит фильтрации по отдельному источнику энергии или модели, а обрабатывает все транспортные средства для построения сводной таблицы и 3D-гистограммы.

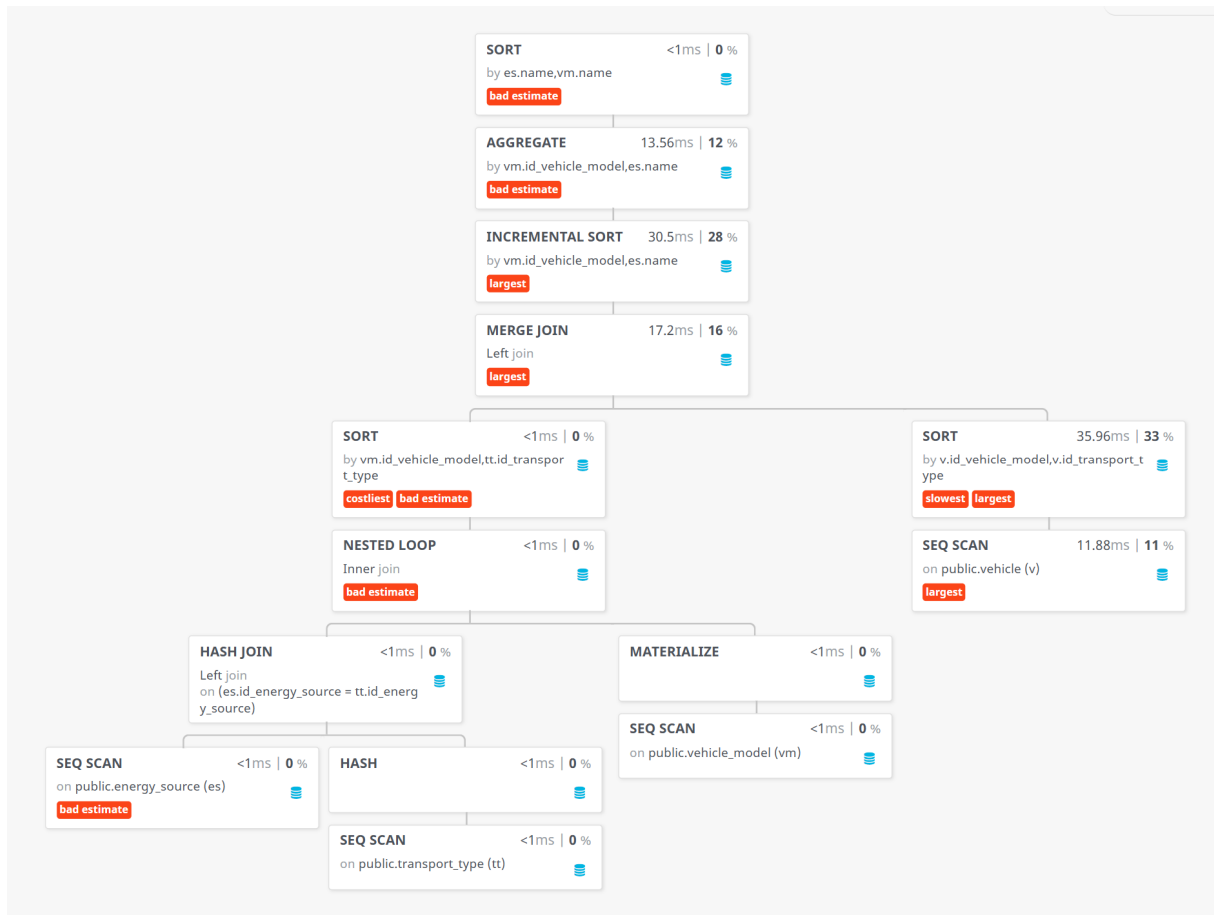


Рис. 38: Графическое представление последовательности выполнения запроса 8

Реляционная формула выполнения запроса:

$$\tau(energy_source, model_name, vehicle_count) \left(\begin{aligned} &\gamma(id_energy_source, id_vehicle_model, \\ &\quad COUNT(id_vehicle) \rightarrow vehicle_count) \\ &((energy_source \times vehicle_model) \bowtie_L transport_type \\ &\quad \bowtie_L vehicle) \end{aligned} \right)$$

Текстовое объяснение последовательности выполнения запроса.

Операция CROSS JOIN между energy_source и vehicle_model формирует все возможные комбинации ($3 \times 9 = 27$ строк). Затем через LEFT JOIN присоединяется transport_type для связи источника энергии с видом транспорта, и далее — vehicle с двойным условием: соответствие

виду транспорта и модели. Это позволяет подсчитать ТС только для корректных комбинаций.

COUNT(id_vehicle) возвращает 0 для комбинаций без ТС. Итого: Diesel — ок. 3692 ТС на модель, Electric overhead wire — ок. 1642, Electric rail current — ок. 1292.

3D-гистограмма распределения ТС по источникам энергии и моделям представлена на рис. 39.

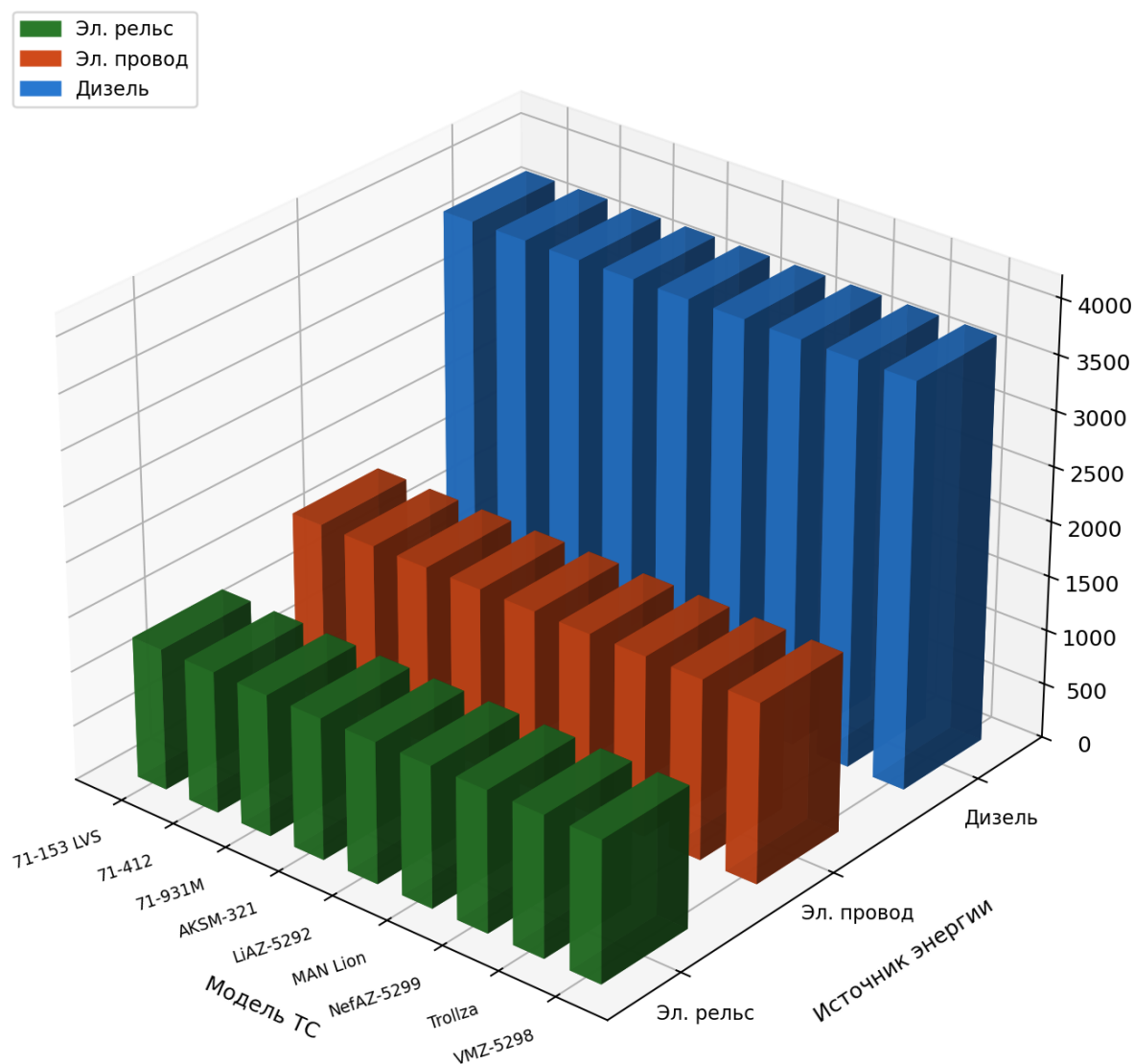


Рис. 39: 3D-гистограмма — Количество ТС по источникам энергии и моделям (запрос 8)

4.9 Запрос 9

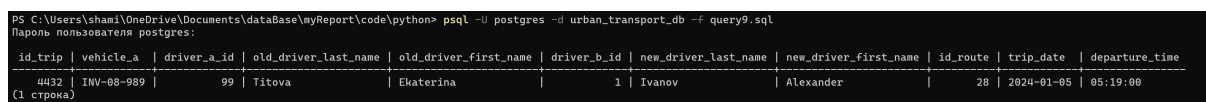
Для транспортного средства А (INV-08-989) в рейсе А (`id_trip = 4432`) поменять водителя А (`id_driver = 99`) на водителя Б (`id_driver = 1`).

Исходный код запроса на языке SQL представлен в листинге 9.

```
1      UPDATE trip t
2      SET id_driver = 1
3      FROM vehicle v
4      WHERE t.id_vehicle = v.id_vehicle
5      AND v.inventory_number = 'INV-08-989'
6      AND t.id_trip = 4432
7      AND t.id_driver = 99
8      RETURNING
9      t.id_trip,
10     v.inventory_number AS vehicle,
11     99 AS old_driver_id,
12     t.id_driver AS new_driver_id,
13     t.id_route,
14     t.trip_date,
15     t.departure_time;
```

Листинг 9: Текст запроса 9 на SQL

Запрос выполняет прямое обновление таблицы `trip`: у рейса 4432 транспортного средства INV-08-989 значение `id_driver` меняется с 99 на 1. Соединение `FROM vehicle v` позволяет идентифицировать ТС по инвентарному номеру. Условие `WHERE` проверяет одновременно идентификатор рейса, инвентарный номер ТС и текущего водителя. Предложение `RETURNING` возвращает обновлённую запись с идентификаторами старого и нового водителей, маршрутом, датой и временем отправления. Результат выполнения запроса 9 представлен на рис. 40.



id_trip	vehicle_a	driver_a_id	old_driver_last_name	old_driver_first_name	driver_b_id	new_driver_last_name	new_driver_first_name	id_route	trip_date	departure_time
4432	INV-08-989	99	Titova	Ekaterina	1	Ivanov	Alexander	28	2024-01-05	05:19:00

(1 строка)

Рис. 40: Результат выполнения запроса 9

Время выполнения запроса: менее 1 мс.

План выполнения запроса 9 представлен на рис. 41. Запрос выполняет операцию `UPDATE` над таблицей `trip`. Сначала по первичному ключу `trip_pkey` выполняется индексное сканирование таблицы `trip` с условием `id_trip = 4432`; затем дополнительно проверяется текущий водитель `id_driver = 99`. Параллельно по уникальному индексу

`vehicle_inventory_number_key` находится транспортное средство с инвентарным номером INV-08-989. После этого узел Nested Loop Inner Join проверяет соответствие `t.id_vehicle = v.id_vehicle`, и найденная строка таблицы `trip` обновляется.

Graphical Analysis Statistics			
#	Node	Rows	Plan
1.	→ Update on public.trip as t (cost=0.71..16.76 rows=1 width=27)	1	
2.	→ Nested Loop Inner Join (cost=0.71..16.76 rows=1 width=27) Join Filter: (t.id_vehicle = v.id_vehicle)	1	
3.	→ Index Scan using trip_pkey on public.trip as t (cost=0.29..8.31 rows=1 width=10) Filter: (t.id_driver = 99) Index Cond: (t.id_trip = 4432)	1	
4.	→ Index Scan using vehicle_inventory_number_key on public.vehicle as v (cost=0.41..8.43 rows=1 width=21) Index Cond: ((v.inventory_number)::text = 'INV-08-989'::text)	1	

Рис. 41: План выполнения запроса 9

Графическое представление последовательности выполнения запроса 9 представлено на рис. 42. На диаграмме видно, что запрос затрагивает одну изменяемую строку таблицы `trip`. Основная работа выполняется двумя индексными сканированиями: первое быстро находит нужный рейс по идентификатору, второе - нужное транспортное средство по инвентарному номеру. После соединения найденных строк выполняется изменение значения `id_driver`, а предложение `RETURNING` возвращает результат обновления.

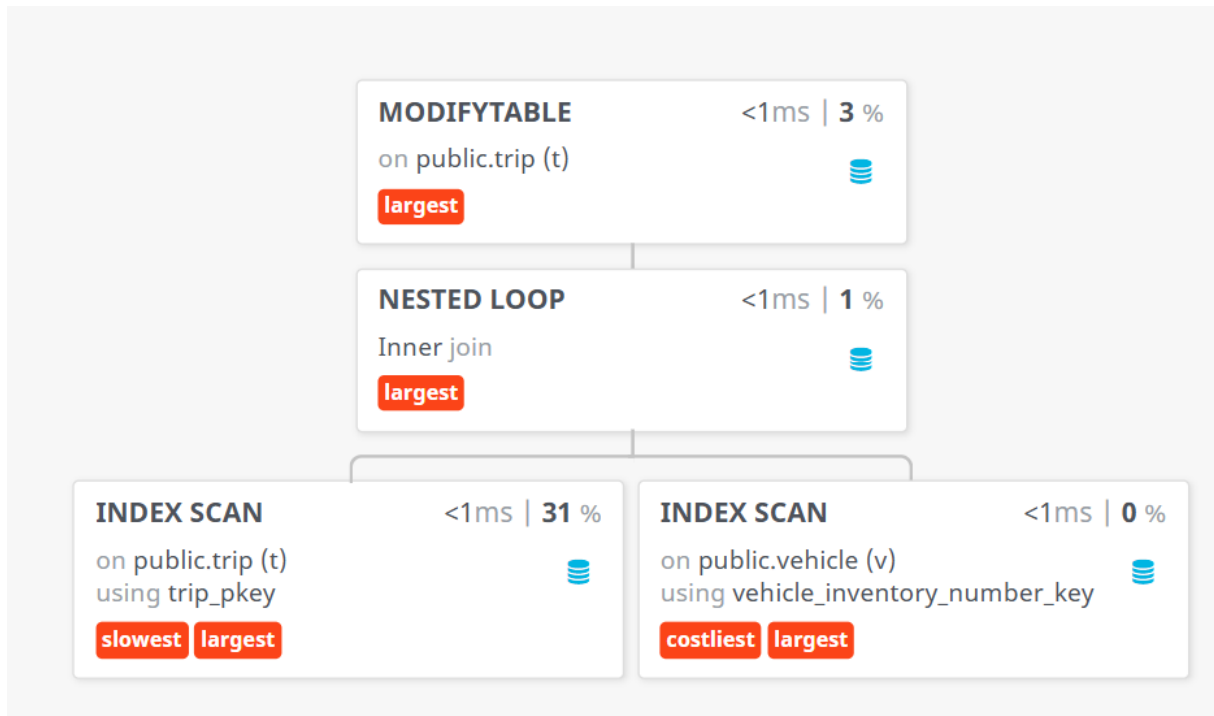


Рис. 42: Графическое представление последовательности выполнения запроса 9

Реляционная формула выполнения запроса:

$$\begin{aligned}
 &\sigma_{id_trip=4432 \wedge inventory_number='INV-08-989'}(trip) \\
 &\quad \wedge id_driver=99 \\
 &:= \{id_driver \leftarrow 1\}
 \end{aligned}$$

Текстовое объяснение последовательности выполнения запроса.

Запрос использует синтаксис `UPDATE ... FROM` для соединения таблицы `trip` с `vehicle` по идентификатору ТС. Условие `WHERE` одновременно фиксирует три параметра: инвентарный номер ТС А (INV-08-989), идентификатор рейса (4432) и текущего водителя А (`id_driver = 99`). Это гарантирует обновление ровно одной строки. Предложение `SET id_driver = 1` устанавливает нового водителя Б. Предложение `RETURNING` возвращает обновлённую строку, фиксируя значение 99 как `old_driver_id` и обновлённое `t.id_driver` как `new_driver_id`.

Заключение

В результате выполнения курсовой работы была спроектирована и реализована реляционная база данных для учёта движения городского общественного транспорта. В ходе работы была проведена анализ предметной области, выделены основные сущности: транспортные средства, водители, маршруты, остановки, рейсы, а также классификационные сущности (вид транспорта, источник энергии, модель ТС). На основе анализа построена ER-диаграмма в нотации Чена и схема связей сущностей, включающих **9 сущностей**.

Разработанная реляционная схема базы данных включает **9 таблиц**, реализованных в СУБД PostgreSQL. Таблицы заполнены тестовыми данными при помощи скрипта на языке Python, реализованного исключительно на стандартной библиотеке (модули `random`, `datetime`, `pathlib`, `argparse` и `subprocess`). Скрипт формирует файл `generated_data.sql` с командами INSERT и при необходимости загружает его в PostgreSQL через утилиту `psql`. База данных содержит свыше **59 000 транспортных средств** и **90 000 рейсов**.

Были разработаны и выполнены **9 SQL-запросов**, охватывающих: выборку с многотабличными соединениями (JOIN), группировку и агрегирование (GROUP BY, COUNT), операции над множествами (UNION ALL), коррелированные подзапросы (NOT EXISTS), оконные функции (COUNT(*) OVER()), а также операцию обновления данных (UPDATE ... RETURNING). Для запросов 3 и 4 построены гистограммы распределения ТС и водителей; для запроса 8 — трёхмерная гистограмма распределения ТС по источникам энергии и моделям.

Для создания ER-диаграмм и схем использовалось веб-приложение «draw.io». Для управления базой данных использовалась система «PostgreSQL». Запросы выполнены и отлажены с помощью командной строки `psql`.

Список использованной литературы

- [1] MySQL Documentation. Data Types. URL: <https://dev.mysql.com/doc/refman/8.0/en/data-types.html>
- [2] MySQL Documentation. CHAR and VARCHAR Types. URL: <https://dev.mysql.com/doc/refman/8.0/en/char.html>
- [3] PostgreSQL Documentation. Data Types. URL: <https://www.postgresql.org/docs/current/datatype.html>
- [4] Stack Overflow. What is the optimum length for a varchar field? URL: <https://stackoverflow.com/questions/1710193/what-is-the-optimum-length-for-a-varchar-field>
- [5] PGFPlots Package Documentation. URL: <https://pgfplots.sourceforge.net/pgfplots.pdf>

Приложение А. Исходный код программы генерации схемы базы данных

```
1      -- Topic: Urban Transport Movement Tracking
2      -- DBMS: PostgreSQL
3
4      DROP TABLE IF EXISTS TRIP CASCADE;
5      DROP TABLE IF EXISTS ROUTE_STOP CASCADE;
6      DROP TABLE IF EXISTS VEHICLE CASCADE;
7      DROP TABLE IF EXISTS TRANSPORT_TYPE CASCADE;
8      DROP TABLE IF EXISTS ENERGY_SOURCE CASCADE;
9      DROP TABLE IF EXISTS VEHICLE_MODEL CASCADE;
10     DROP TABLE IF EXISTS DRIVER CASCADE;
11     DROP TABLE IF EXISTS ROUTE CASCADE;
12     DROP TABLE IF EXISTS STOP CASCADE;
13
14     CREATE TABLE ENERGY_SOURCE (
15     id_Energy_Source SERIAL PRIMARY KEY,
16     Name VARCHAR(50) NOT NULL UNIQUE
17     );
18
19     CREATE TABLE VEHICLE_MODEL (
20     id_Vehicle_Model SERIAL PRIMARY KEY,
21     Name VARCHAR(100) NOT NULL
22     );
23
24     CREATE TABLE TRANSPORT_TYPE (
25     id_Transport_Type SERIAL PRIMARY KEY,
26     Description VARCHAR(200),
27     Infrastructure_Type VARCHAR(100) NOT NULL,
28     id_Energy_Source INT NOT NULL
29     REFERENCES ENERGY_SOURCE(id_Energy_Source),
30     Capacity INT NOT NULL CHECK (
31         Capacity > 0)
32     );
33
34     CREATE TABLE DRIVER (
35     id_Driver SERIAL PRIMARY KEY,
36     Last_Name VARCHAR(100) NOT NULL,
37     First_Name VARCHAR(100) NOT NULL,
38     Middle_Name VARCHAR(100),
39     License_Category VARCHAR(10) NOT NULL
40     );
41
42     CREATE TABLE ROUTE (
43     id_Route SERIAL PRIMARY KEY,
44     Route_Name VARCHAR(100) NOT NULL,
```

```

44     Length_km    DECIMAL(6,2)    NOT NULL CHECK (Length_km >
        0),
45     Route_Type   VARCHAR(50)      NOT NULL,
46     Status       VARCHAR(20)      NOT NULL DEFAULT 'Active'
47 );
48
49 CREATE TABLE STOP (
50     id_Stop       SERIAL           PRIMARY KEY,
51     Stop_Name     VARCHAR(200)    NOT NULL,
52     Stop_Address  VARCHAR(200),
53     Stop_Type     VARCHAR(50)     NOT NULL,
54     District      VARCHAR(100),
55     Coordinates   VARCHAR(50)
56 );
57
58 CREATE TABLE VEHICLE (
59     id_Vehicle     SERIAL           PRIMARY KEY,
60     id_Vehicle_Model INT           NOT NULL
61     REFERENCES VEHICLE_MODEL(id_Vehicle_Model),
62     Capacity       INT             NOT NULL CHECK (
        Capacity > 0),
63     Inventory_Number VARCHAR(20)   NOT NULL UNIQUE,
64     Year_of_Manufacture INT         NOT NULL
65     CHECK (Year_of_Manufacture BETWEEN 1990 AND 2026),
66     id_Transport_Type INT          NOT NULL
67     REFERENCES TRANSPORT_TYPE(id_Transport_Type)
68 );
69
70 CREATE TABLE ROUTE_STOP (
71     id_Route_Stop  SERIAL           PRIMARY KEY,
72     id_Route       INT             NOT NULL
73     REFERENCES ROUTE(id_Route),
74     id_Stop        INT             NOT NULL
75     REFERENCES STOP(id_Stop),
76     Stop_Order     INT             NOT NULL CHECK (
        Stop_Order > 0),
77     Distance_From_Start DECIMAL(6,2) NOT NULL
78     CHECK (Distance_From_Start >= 0),
79     Arrival_Time   TIME,
80     UNIQUE (id_Route, Stop_Order)
81 );
82
83 CREATE TABLE TRIP (
84     id_Trip        SERIAL           PRIMARY KEY,
85     Trip_Date      DATE            NOT NULL,
86     Departure_Time TIME            NOT NULL,
87     Arrival_Time   TIME,

```

```

86      Status          VARCHAR(20)  NOT NULL DEFAULT '
      Completed',
87      id_Vehicle       INT           NOT NULL REFERENCES
      VEHICLE(id_Vehicle),
88      id_Driver        INT           NOT NULL REFERENCES
      DRIVER(id_Driver),
89      id_Route         INT           NOT NULL REFERENCES
      ROUTE(id_Route)
90  );
91
92  CREATE INDEX idx_trip_date      ON TRIP(Trip_Date);
93  CREATE INDEX idx_trip_route    ON TRIP(id_Route);
94  CREATE INDEX idx_trip_vehicle  ON TRIP(id_Vehicle)
95  ;
96  CREATE INDEX idx_trip_driver   ON TRIP(id_Driver);
97  CREATE INDEX idx_route_stop_route ON ROUTE_STOP(
      id_Route);
      CREATE INDEX idx_vehicle_type ON VEHICLE(
      id_Transport_Type);

```

Листинг 10: Исходный код создания схемы базы данных (PostgreSQL)

Приложение Б. Исходный код программы заполнения базы данных текстовыми данными

```
1      """
2      Generate test data for the Urban Transport database.
3      Uses only the Python standard library (no third-party
4      packages).
5      Writes INSERT statements to generated_data.sql;
6      optionally
7      executes it with psql (--execute flag).
8
9      Default quantities:
10     3 energy sources | 9 vehicle models | 20 transport
11     types
12     2000 drivers | 100 routes | 200 stops
13     2800-3200 vehicles per transport type (~60 000 total
14     )
15     43-47 trips per driver (~90 000 total
16     )
17     Total records: more than 150 000
18     """
19
20     from __future__ import annotations
21     import argparse, datetime as dt, random, subprocess
22     from pathlib import Path
23
24     ENERGY_SOURCES = ["Diesel", "Electric overhead wire",
25     "Electric rail current"]
26     VEHICLE_MODELS = [
27     "LiAZ-5292", "NefAZ-5299", "MAN Lion's City",
28     "71-931M Vityaz", "71-153 LVS-2005", "71-412 Pchyolka
29     ",
30     "VMZ-5298.01", "AKSM-321", "Trollza-5265",
31     ]
32     DRIVER_COUNT = 2000
33     ROUTE_COUNT = 100
34     STOP_COUNT = 200
35     VEHICLES_PER_TYPE_MIN = 2800
36     VEHICLES_PER_TYPE_MAX = 3200
37     TRIPS_PER_DRIVER_MIN = 43
38     TRIPS_PER_DRIVER_MAX = 47
39
40     def sql_text(value: str | None) -> str:
41         if value is None:
42             return "NULL"
43         return "'" + value.replace("'", "''") + "'"
44
45     parser = argparse.ArgumentParser(
46         description="Generate test data for the Urban Transport database.",
47         epilog="Writes INSERT statements to generated_data.sql; optionally"
48         "executes it with psql (--execute flag).",
49         formatter_class=argparse.RawTextHelpFormatter,
50     )
51     parser.add_argument(
52         "-c", "--count", type=int, default=100,
53         help="Number of records to generate (default: 100).",
54     )
55     parser.add_argument(
56         "-e", "--execute", action="store_true",
57         help="Execute the generated SQL statements using psql.",
58     )
59     parser.add_argument(
60         "-p", "--psql", type=str, default="psql",
61         help="psql command (default: psql).",
62     )
63     parser.add_argument(
64         "-s", "--seed", type=int, default=42,
65         help="Random seed (default: 42).",
66     )
67     parser.add_argument(
68         "-t", "--types", type=str, default="all",
69         help="Types of data to generate (default: all).",
70     )
71     parser.add_argument(
72         "-v", "--verbose", action="store_true",
73         help="Verbose output.",
74     )
75     args = parser.parse_args()
76     count = args.count
77     execute = args.execute
78     psql = args.psql
79     seed = args.seed
80     types = args.types
81     verbose = args.verbose
82
83     random.seed(seed)
84
85     # Generate data
86     energy_sources = random.sample(ENERGY_SOURCES, count)
87     vehicle_models = random.sample(VEHICLE_MODELS, count)
88     drivers = random.sample(range(1, count + 1), count)
89     routes = random.sample(range(1, count + 1), count)
90     stops = random.sample(range(1, count + 1), count)
91     vehicles_per_type = random.sample(
92         range(VEHICLES_PER_TYPE_MIN, VEHICLES_PER_TYPE_MAX + 1), count)
93     trips_per_driver = random.sample(
94         range(TRIPS_PER_DRIVER_MIN, TRIPS_PER_DRIVER_MAX + 1), count)
95
96     # Write to file
97     with open("generated_data.sql", "w") as f:
98         for i in range(count):
99             f.write(
100                 f"INSERT INTO energy_sources (source) VALUES ('{energy_sources[i]}');\n"
101                 f"INSERT INTO vehicle_models (model) VALUES ('{vehicle_models[i]}');\n"
102                 f"INSERT INTO drivers (driver) VALUES ({drivers[i]});\n"
103                 f"INSERT INTO routes (route) VALUES ({routes[i]});\n"
104                 f"INSERT INTO stops (stop) VALUES ({stops[i]});\n"
105                 f"INSERT INTO vehicles_per_type (type, count) VALUES ({types[i]}, {vehicles_per_type[i]});\n"
106                 f"INSERT INTO trips_per_driver (driver, trips) VALUES ({drivers[i]}, {trips_per_driver[i]});\n"
107             )
108
109     # Execute
110     if execute:
111         subprocess.run(psql + " -f generated_data.sql", shell=True)
```

```

39     def insert_many(table: str, columns: list[str],
40                     values: list[str], batch_size: int = 500) -> str:
41         col = ", ".join(columns)
42         parts = []
43         for i in range(0, len(values), batch_size):
44             parts.append(
45                 f"INSERT INTO {table} ({col}) VALUES\n"
46                 + ",\n".join(values[i:i + batch_size]) + ";" )
47         return "\n\n".join(parts)
48
49     def build_sql(seed: int, start: dt.date,
50                  end: dt.date) -> tuple[str, dict[str, int]]:
51         rng = random.Random(seed)
52         lines: list[str] = [
53             "-- Generated by fill_database.py (standard library
54              only)",
55             "TRUNCATE TABLE TRIP, ROUTE_STOP, VEHICLE,",
56             "    TRANSPORT_TYPE, ENERGY_SOURCE, VEHICLE_MODEL,",
57             "    DRIVER, ROUTE, STOP RESTART IDENTITY CASCADE;",
58             "",
59             ]
60         lines.append(insert_many("ENERGY_SOURCE", ["Name"],
61                                 [f"({sql_text(n)})" for n in ENERGY_SOURCES]))
62         lines.append(insert_many("VEHICLE_MODEL", ["Name"],
63                                 [f"({sql_text(n)})" for n in VEHICLE_MODELS]))
64         # Drivers, routes, stops, vehicles, route_stops and
65         # trips
66         # are generated in the same pattern using rng and
67         # insert_many.
68         # Full source: fill_database.py
69         return "\n\n".join(lines) + "\n", {}
70
71     def main() -> None:
72         p = argparse.ArgumentParser()
73         p.add_argument("--seed", default=20260512, type=int)
74         p.add_argument("--start-date", default="2024-01-01")
75         p.add_argument("--end-date", default="2026-05-12")
76         p.add_argument("--output", default="generated_data.sql")
77         p.add_argument("--execute", action="store_true")
78         args = p.parse_args()
79         start = dt.date.fromisoformat(args.start_date)
80         end = dt.date.fromisoformat(args.end_date)
81         out = Path(args.output)
82         sql, _ = build_sql(args.seed, start, end)
83         out.write_text(sql, encoding="utf-8")
84         print(f"Generated SQL: {out}")

```



```
81         if args.execute:
82             subprocess.run(["psql", "-f", str(out)], check=True)
83         print("Data loaded successfully.")
84
85     if __name__ == "__main__":
86         main()
```

Листинг 11: Фрагмент кода генерации тестовых данных (Python)